

SĂPTĂMÂNA 8

Automate celulare, particule, simulare de înaltă performanță

Modelare și Simulare · Anul III, Calculatoare și Tehnologia Informației

Ș.l. dr. ing. Cătălin Iapă

Unde am rămas

- ✓ Sisteme continue: ecuații diferențiale rezolvate cu Euler, **RK4**, solve_ivp (S7)
- ✓ Euler simplectic păstrează orbitele închise – metodele **Verlet** de azi sunt rudele lui
- ✓ SIR și Lotka–Volterra: populații **omogen amestecate**, fără spațiu
- ✓ Întrebarea de final: ce facem cu **un milion** de particule sau o grilă cu un miliard de celule?

La finalul acestei săptămâni veți putea…

1. să definiți un **automat celular**: rețea, stări, vecinătate, regulă, actualizare sincronă
2. să simulați automate **1D elementare** (regulile 30, 90, 110, 184) și **Game of Life**, vectorizat cu NumPy
3. să explicați **emergența** și clasele de comportament ale lui Wolfram
4. să construiți modele CA pentru **trafic** (Nagel–Schreckenberg), incendii, difuzie și să explicați ideea **Lattice Boltzmann**
5. să integrați un sistem de **particule** cu Verlet și să reduceți costul de la $O(N^2)$ la $O(N)$ sau $O(N \log N)$
6. să estimați câștigul din **vectorizare** și **paralelism** (legea lui Amdahl) și ce nu se poate paraleliza

Ecuații diferențiale vs automate celulare

Ecuații diferențiale (S7)

- stare **continuă**: temperaturi, concentrații
- timp continuu, aproximat cu pași h
- mărimi agregate, de obicei fără spațiu
- ecuații derivate din fizică sau bilanțuri

vs

Automate celulare (S8)

- **totul** este discret: spațiu, timp, stări
- fiecare celulă urmează o **regulă locală**
- spațiul este esențial: cine este vecinul cui
- comportamentul global **emerge** din reguli simple

Cernazean (L4): un automat celular este „un univers fictiv în care totul este discret” – și, în același timp, un model matematic și o paradigmă de calcul. Pe calculator, ambele lumi devin pași discreți; diferența este ce discretizăm și ce păstrăm.

Unde apar automatele celulare

1 Fizică

Fluide (Lattice Boltzmann), creșterea cristalelor, magnetism, difuzie.

2 Biologie

Diferențierea celulelor, unde în țesuturi excitabile, tumori, ecosisteme.

3 Trafic și orașe

Modelul Nagel–Schreckenberg, simulatoare de trafic urban, evacuări.

4 Mediu

Incendii de pădure, inundații, răspândirea speciilor invazive.

5 Informatică

Generatoare de numere aleatoare, criptografie, procesarea imaginilor, generare procedurală în jocuri.

6 Hardware

Calcul paralel natural: fiecare celulă se actualizează independent – ideal pentru GPU.

Ora 1

Automate celulare

1. Definiție: rețea, stări, vecinătate, regulă
2. Istoric: von Neumann, Ulam și autoreproducerea
3. Automatele 1D elementare și clasele lui Wolfram
4. Game of Life, emergență, furnica lui Langton

Ce este un automat celular

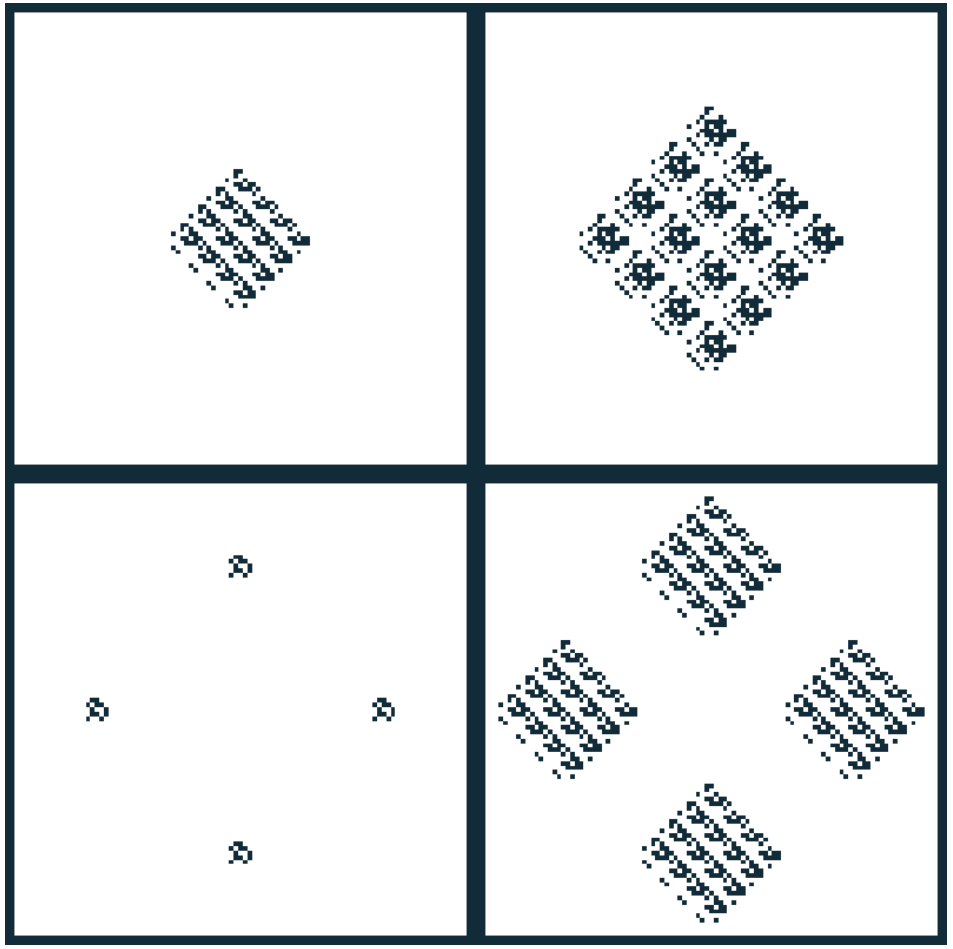
Automatul celular = ⟨A, S, N, Φ⟩

O **rețea** regulată de celule A (în d dimensiuni), o mulțime finită de **stări** S, o **vecinătate** N și o **regulă de evoluție** Φ, aceeași pentru toate celulele. La fiecare pas de timp, toate celulele își calculează **simultan** starea următoare din starea lor și a vecinilor.

Cernazean L4; Chopard & Droz, 1998

- Spațiu discret, timp discret, stări discrete (Cernazean L4)
- Regula este **locală** (doar vecinii) și **omogenă** (aceeași peste tot)
- Actualizarea sincronă: starea nouă se calculează din starea veche a tuturor – avem nevoie de o copie
- Starea inițială și condițiile la frontieră completează modelul

Regula parității: suma celor 4 vecini, modulo 2



pornind de la 12 celule; pașii 13, 27, 32, 45 (stânga–dreapta, sus–jos)

- Stări 0/1, vecinătate von Neumann
- Regula: 1 dacă suma vecinilor e **impară**
- Modele complicate din reguli simple
- La pasul 32: **48** = 4 × 12 celule – patru **copii** ale modelului inițial

Vecinătăți: cine contează pentru o celulă

von Neumann (4 vecini)

- nord, sud, est, vest
- distanța „Manhattan” 1
- difuzie, regula parității, gaze pe rețea
- informația se propagă în **romb**

Moore (8 vecini)

- și cele 4 diagonale
- distanța „șah” 1 (regele)
- Game of Life, medii excitabile
- informația se propagă în **pătrat**

Alte vecinătăți: **Margolus** (blocuri de 2 × 2, alternând pozițiile – util pentru reguli reversibile și gaze), **hexagonală** (6 vecini, mai izotropă: fulgi de zăpadă, fluide FHP), vecinătăți cu rază mai mare. Forma vecinătății lasă urme în formele care apar.

Condiții la frontieră (Cernazean L4)

Tipul	Ce se întâmplă la margine	Când îl folosim
Periodic	grila se închide ca un tor: vecinul ultimei celule este prima	sisteme „infinite”, fără efecte de margine
Fix	celulele din afară au o valoare constantă (de ex. 0)	pereți, surse, rezervoare
Reflexiv	starea din afară este oglinda celei din interior	simetrii, pereți care reflectă
Adiabatic	celula din afară copiază celula de la margine	fără flux prin frontieră (difuzie)

- În NumPy, np.roll implementează direct condiția **periodică**; celelalte cer o „coroană” de celule fantomă (padding)
- Frontiera poate schimba complet rezultatul: un ambuteiaj pe un inel nu dispare niciodată, pe un drum deschis poate ieși din sistem

Implementarea regulii: NumPy și tabel de căutare

```
import numpy as np

def parity_step(a):
    """Regula paritățiii, condiții periodice."""
    s = (np.roll(a, 1, 0) + np.roll(a, -1, 0) +
         np.roll(a, 1, 1) + np.roll(a, -1, 1))
    return s % 2

# varianta cu tabel de căutare (Cernazean L4):
# index = s_N + 2*s_E + 4*s_S + 8*s_V
# a_nou = regula[index]   (16 intrări)
```

- np.roll deplasează **toată** grila deodată
- Fără bucle Python: totul în cod compilat
- Tabelul de căutare: orice regulă, același cod
- Copia este implicită: s se calculează din a vechi

Câte universuri posibile?

$$\textit{numărul de reguli} = m^{(m^k)}$$

m, k	m stări pe celulă, k celule în vecinătate (inclusiv celula însăși): există m^k configurații locale
1D elementar	m = 2, k = 3: $2^{(2^3)} = 2^8 =$ **256** de reguli – le putem studia pe toate
Moore 2D	m = 2, k = 9: 2^512 ≈ **$1{,}3 \cdot 10^{154}$** reguli – mai multe decât atomii din universul observabil (~10^{80})

Cernazean: „cele mai multe sunt neinteresante”. Game of Life este o regulă particulară, aleasă de Conway după multe încercări, care stă la granița dintre ordine și haos.

Istoric

Von Neumann, Ulam și mașinile care se reproduc

1. Anii 1940: logica autoreproducerii
2. Bucla lui Langton (1984)
3. De la Conway la Wolfram

Repere în istoria automatelor celulare

von Neumann și **Ulam**: un automat cu 29 de stări care se reproduce

1948–53

1970

Game of Life (J. H. Conway), popularizat de M. Gardner în Scientific American

Wolfram: automatele elementare și cele 4 clase; **Langton**: bucla cu 8 stări

1983–84

1986

FHP: gaz pe rețea hexagonală care reproduce ecuațiile fluidelor

Nagel–Schreckenberg: modelul de trafic cu ambuteiaje spontane

1992

2004

M. Cook demonstrează că regula 110 este un calculator universal

Azi: automate cu stări continue (Lenia), automate „neurale” care învață regula din date, simulări pe GPU cu miliarde de celule.

Autoreproducerea: de la von Neumann la Langton

- Întrebarea lui von Neumann: poate o mașină să construiască o copie a ei, inclusiv a „planului” după care se construiește?
- Soluția lui: un **constructor universal** + o **descriere** (bandă) care este și **executată**, și **copiată** – înainte de descoperirea structurii ADN (1953)
- Implementarea: un automat cu **29 de stări** și zeci de mii de celule; s-a dovedit și calculator universal
- **Langton** (1984): o buclă cu doar 8 stări care se reproduce, dar nu mai este calculator universal
- Lecția (Cernazean): autoreproducerea poate fi descrisă **mecanic**, fără o ierarhie în care „complicatul construiește simplul”

A **Legătura cu biologia**

Aceeași dublă folosire a informației – citită ca instrucțiuni și copiată ca date – apare în celulele vii: ADN-ul este transcris și replicat.

Cel mai simplu caz

Automatele 1D elementare

1. 256 de reguli și numerotarea lui Wolfram
2. Regulile 30, 90, 110, 184
3. Cele patru clase de comportament

Automatele elementare și numerotarea Wolfram

3 celule, 2 stări: 8 configurații, 256 de reguli

Fiecare celulă privește la ea și la cei doi vecini: 8 configurații posibile (111, 110, …, 000). Regula dă starea nouă pentru fiecare; cei 8 biți, citiți ca număr binar, dau **numărul regulii** (Wolfram, 1983).

Wolfram, Statistical mechanics of cellular automata, 1983

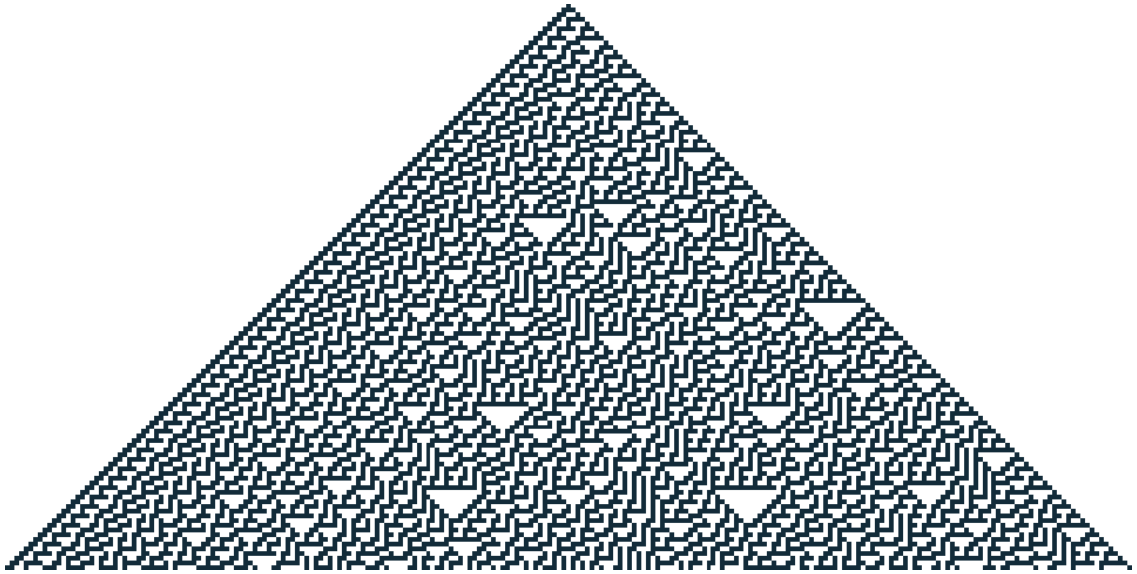
- Regula 30: 00011110 în binar = 30
- Diagrama spațiu–timp: fiecare rând este un pas de timp, de sus în jos
- Implementare: index = 4·stânga + 2·centru + dreapta, apoi tabel[index] – exact ca la regula parității

Patru reguli celebre: tabelele lor

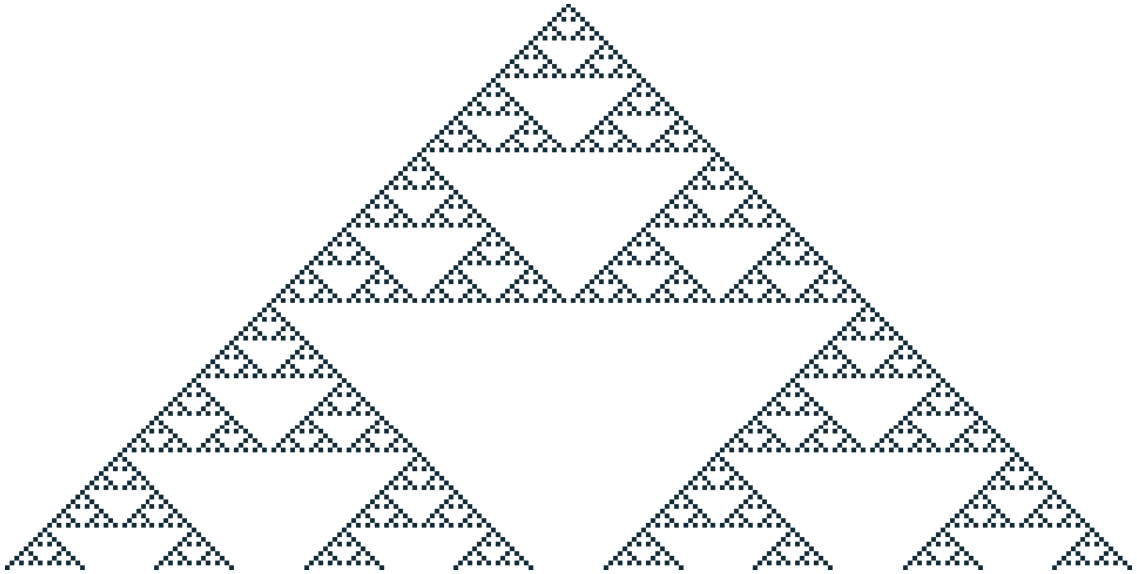
Regula	111	110	101	100	011	010	001	000
30	0	0	0	1	1	1	1	0
90	0	1	0	1	1	0	1	0
110	0	1	1	0	1	1	1	0
184	1	0	1	1	1	0	0	0

- Regula 90: starea nouă = stânga XOR dreapta (triunghiul lui Sierpiński, Pascal modulo 2)
- Regula 184: o mașină (1) înaintează dacă celula din față este liberă – cel mai simplu model de **trafic** (ora 2)

Pornind de la o singură celulă: regula 30 și regula 90

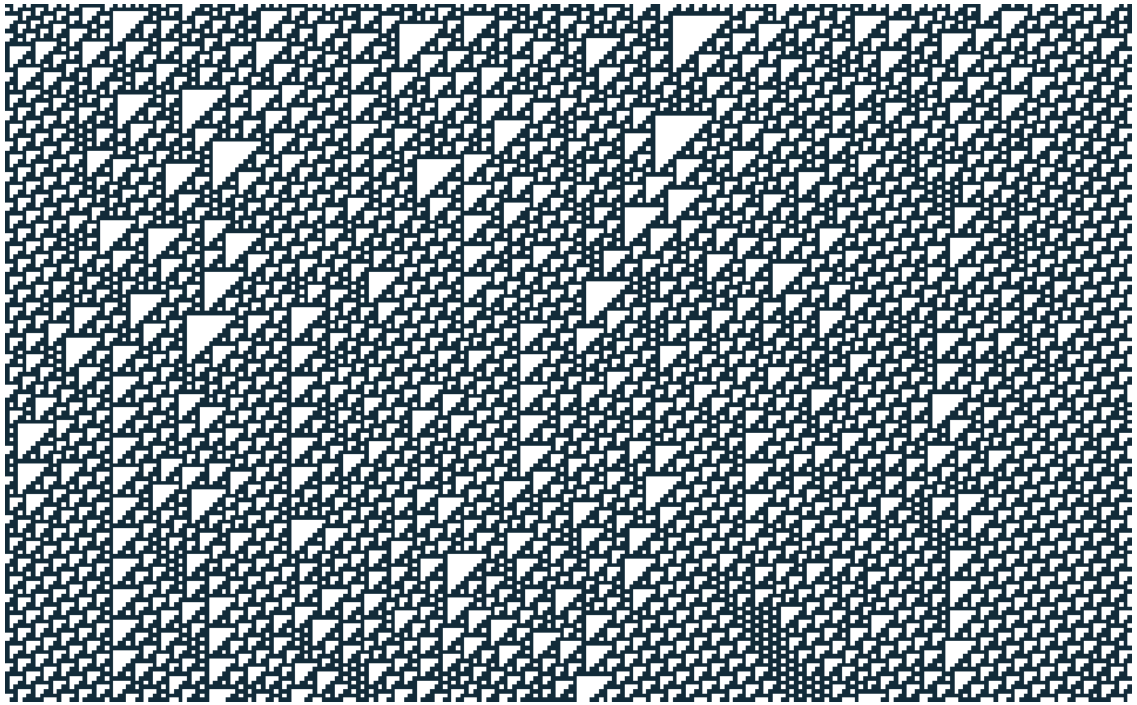


Regula 30: haos – coloana din mijloc pare aleatoare

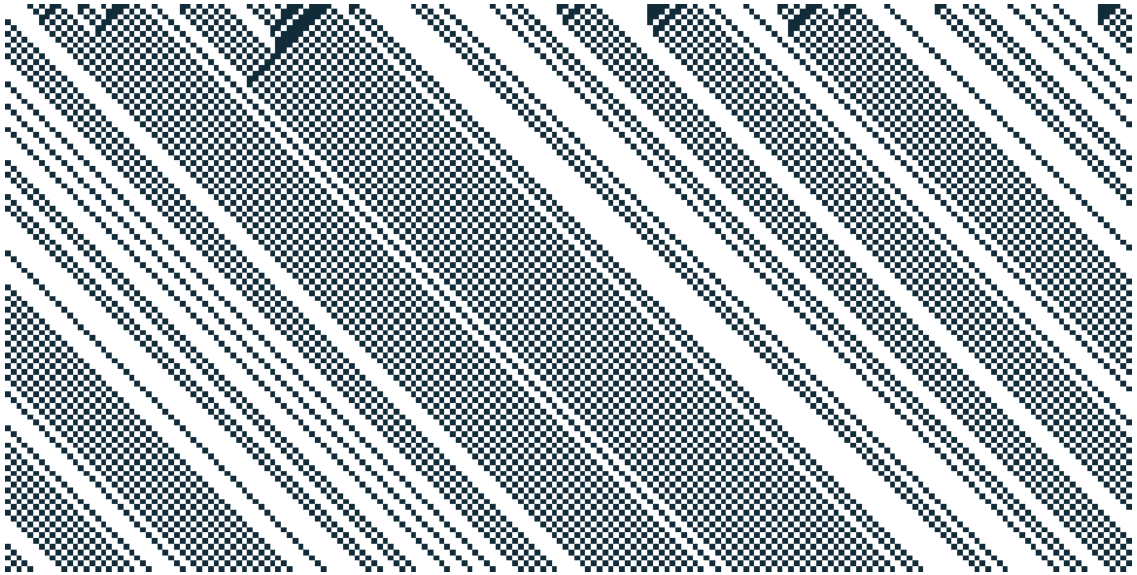


Regula 90: triunghiul lui Sierpiński (fractal)

Din stare inițială aleatoare: regula 110 și regula 184



Regula 110: structuri care se deplasează și interacționează



Regula 184: „mașini” care înaintează, ambuteiaje care se dizolvă

Clasele lui Wolfram (1984)

1 Clasa I – punct fix

Orice stare inițială ajunge la o configurație uniformă (de ex. regula 32: totul devine 0).

2 Clasa II – periodic

Structuri stabile sau oscilante, separate (de ex. regula 108).

3 Clasa III – haos

Modele aperiodice, aparent aleatoare, sensibile la condițiile inițiale (regulile 30, 90).

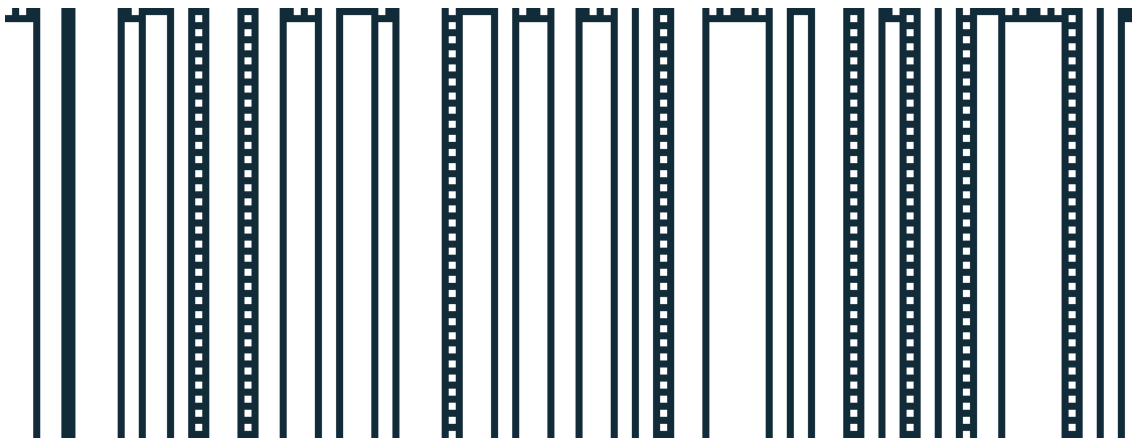
4 Clasa IV – complex

Structuri persistente care se mișcă și interacționează; pot fi calculatoare universale (regula 110, Game of Life).

Clasa I și clasa II, din aceeași stare inițială aleatoare



Regula 32: totul dispare în câțiva pași (clasa I)



Regula 108: structuri fixe și oscilante (clasa II)

De ce sunt celebre aceste reguli

- **Regula 30**: coloana centrală a fost folosită ca generator de numere aleatoare (Mathematica); primii biți: 1101110011000101…
- **Regula 90**: liniară (XOR) – se poate prezice exact: celula (t, x) este coeficientul binomial C(t, ·) modulo 2
- **Regula 110**: calculator **universal** (Cook, 2004) – un automat cu 8 biți de regulă poate calcula orice
- **Regula 184**: trafic, dar și depunerea particulelor, cozi de procese; conservă numărul de 1 (mașinile nu dispar)
- Lecția: complexitatea comportamentului **nu** cere reguli complicate

U **Pentru informaticieni**

Universalitatea regulii 110 înseamnă că întrebarea „va ajunge acest automat vreodată în starea X?” este, în general, **nedecidabilă** – ca problema opririi.

Verificare

? Câte automate celulare **elementare** (1D, 2 stări, vecinătate de 3 celule) există?

A 8

B 30

C 256

D 65 536

Conway, 1970

Game of Life

1. Regulile: naștere cu 3, supraviețuire cu 2 sau 3
2. Planorul și alte „organisme”
3. Implementare vectorizată (demo-ul săptămânii)
4. Life este un calculator universal

Regulile jocului vieții

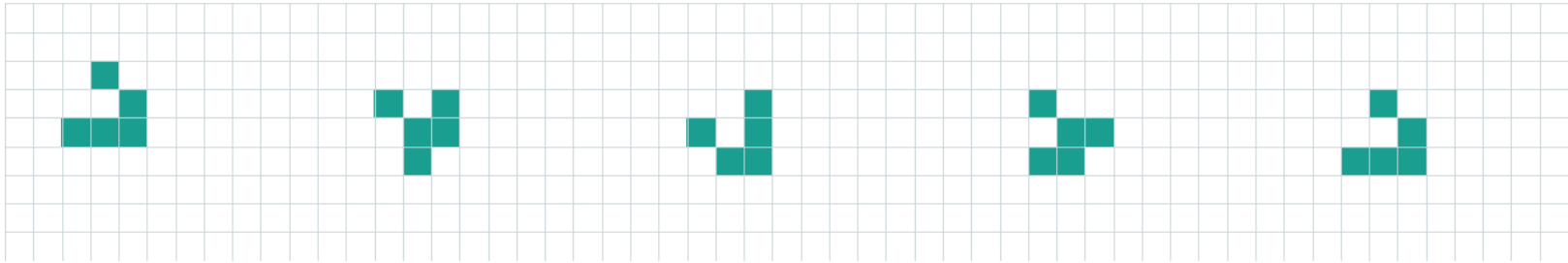
B3/S23: naștere cu 3, supraviețuire cu 2 sau 3

Grilă pătrată, celule **vii** sau **moarte**, vecinătate **Moore** (8 vecini). O celulă moartă cu exact **3** vecini vii **se naște**; o celulă vie cu **2 sau 3** vecini vii **supraviețuiește**; în rest celula moare (de izolare sau de suprapopulare).

Gardner, Scientific American, octombrie 1970; Cernazean L4

- Nu există jucători: „jocul” este doar evoluția din starea inițială
- Conway a căutat o regulă la limita dintre creștere explozivă și stingere
- Notația B/S permite alte variante: HighLife (B36/S23), Seeds (B2/S), peșteri (B5678/S45678)

Planorul (glider): o structură care se deplasează



cinci pași consecutivi; după 4 pași, aceeași formă, mutată cu o celulă pe diagonală

- Perioada **4**, viteza c/4
- Transportă **informație** prin grilă
- Tunul lui Gosper (1970): produce un planor la fiecare 30 de pași
- Cernazean: o „zoologie” de organisme

Demo: Game of Life vectorizat cu NumPy

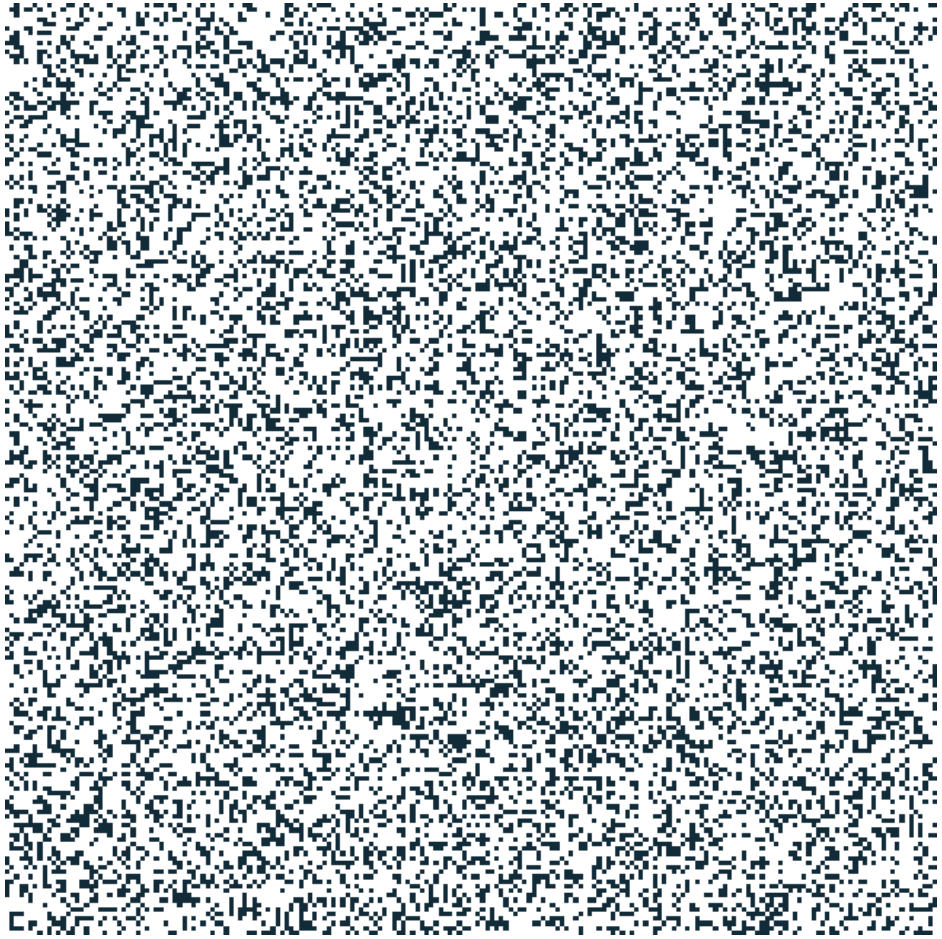
```
import numpy as np

def life_step(a):
    # numărul de vecini vii (Moore, periodic)
    n = sum(np.roll(np.roll(a, i, 0), j, 1)
            for i in (-1, 0, 1) for j in (-1, 0, 1)
            if (i, j) != (0, 0))
    return ((n == 3) | ((a == 1) & (n == 2))).astype(np.uint8)

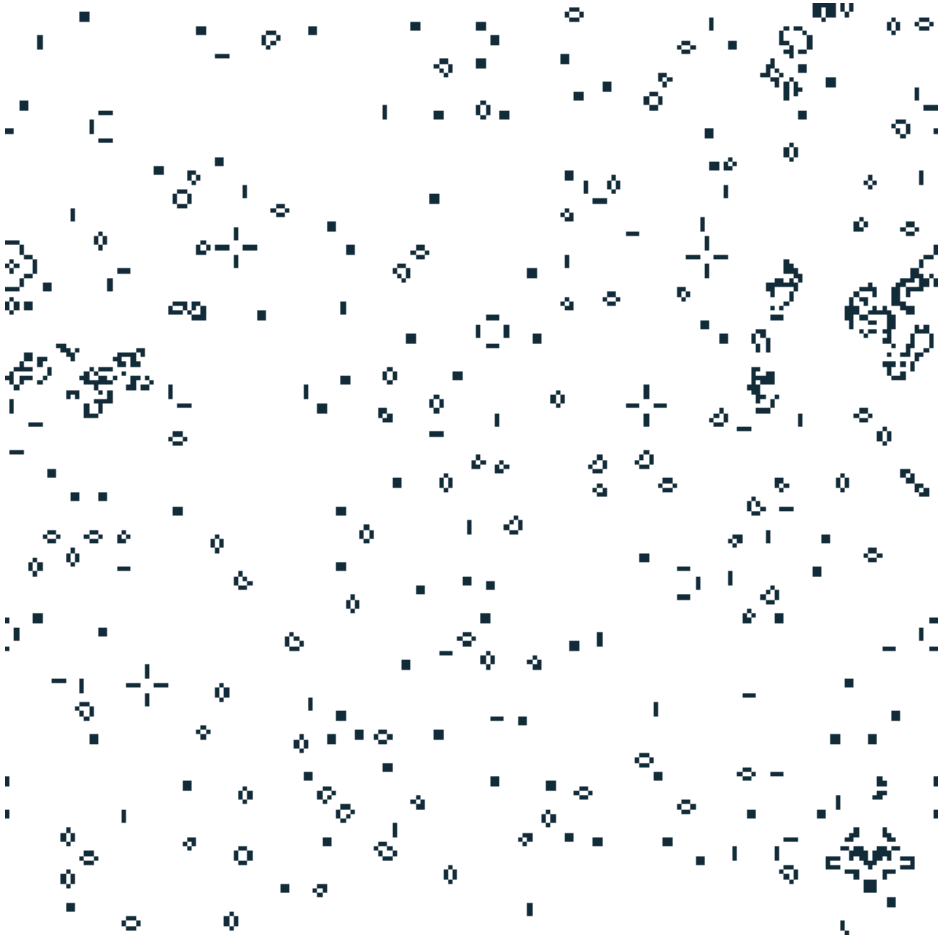
rng = np.random.default_rng(82)
a = (rng.random((200, 200)) < 0.3).astype(np.uint8)
for t in range(1000):
    a = life_step(a)
```

- 8 deplasări ale grilei = numărul de vecini
- Regula: o singură expresie booleană
- Toate celulele **simultan**
- Bucle Python: 150 ms pe pas; NumPy: 0,27 ms

200 × 200 celule, 30% vii la început: pasul 0 și pasul 1 000

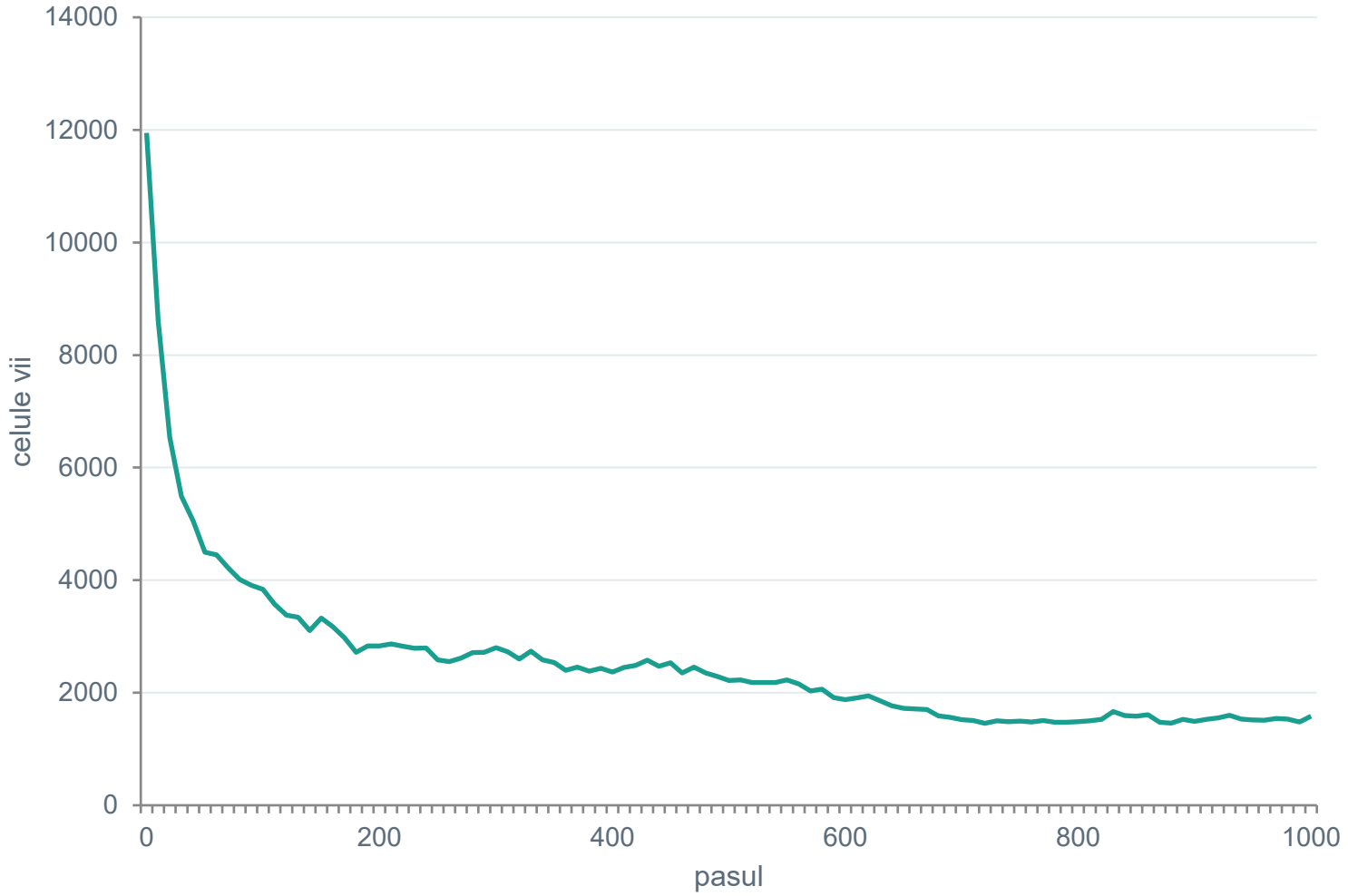


pasul 0: 11 946 celule vii



pasul 1 000: 1 584 celule vii – „cenușă” de structuri stabile și oscilante

Populația în timp: de la haos la „cenușă”



- Start: **11 946** celule vii
- Scădere rapidă în primii ~50 de pași
- Apoi fluctuații lente, pe măsură ce zonele active se sting
- După 1 000 de pași: **1 584** (~4% din grilă)

De ce vectorizăm: un pas din Life pe 256 × 256 celule

150 ms

bucle Python imbricate, celulă cu celulă

0,27 ms

NumPy: 8 deplasări de vectori și o expresie booleană

≈ 557×

mai rapid, cu **același** rezultat (verificat celulă cu celulă)

- Măsurat în mediul nostru de test; ordinul de mărime contează, nu cifra exactă
- Aceeași idee pe GPU (CuPy, JAX): încă un factor de 10–100 pentru grile mari – ora 3

Life este un calculator universal

- Planoarele sunt **semnale**; coliziunile lor pot implementa porți logice AND, OR, NOT
- Berlekamp, Conway și Guy (1982): Life poate simula orice calculator – o **mașină Turing universală**
- P. Rendell (2000): o mașină Turing construită efectiv în Life
- Metapixelul OTCA (2006): o structură care simulează o celulă Life – **Life rulat în Life**
- Consecință: ca la regula 110, întrebări simple despre viitorul unei configurații sunt **nedecidabile**

I **Ireductibilitate**

Pentru multe configurații nu există o scurtătură: singurul mod de a afla ce se întâmplă la pasul 10^6 este să simulezi cei 10^6 pași.

Verificare

? În Game of Life, o celulă **moartă** are exact **3** vecini vii. Ce se întâmplă la pasul următor?

A rămâne moartă

B devine vie (naștere)

C devine vie doar dacă avea 2 vecini la pasul anterior

D depinde de vecinii vecinilor

Când regula nu ajunge

Emergență și ireductibilitate

1. Furnica lui Langton
2. Medii excitabile: unde și spirale
3. Variante: stocastic, asincron, continuu, „neural”

Furnica lui Langton: 10 000 de pași de haos, apoi o „autostradă”



pasul 5 000: o pată haotică



pasul 11 000: furnica a început să construiască o autostradă

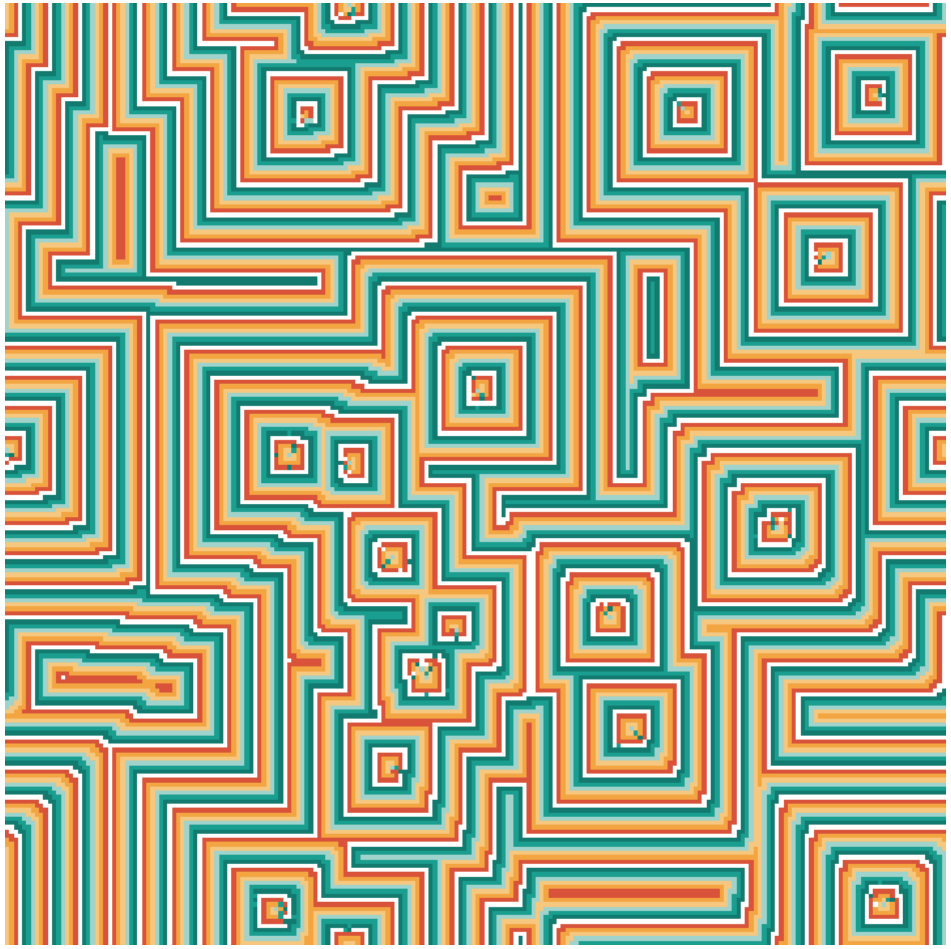
Lecția furnicii pentru metodologia științifică (Cernazean L4)

- Cunoaștem **perfect** legea fundamentală a sistemului – pentru că noi am definit-o
- Totuși **nu putem prezice** detaliile: când apare autostrada, unde se îndreaptă
- Descrierea microscopică nu prezice întotdeauna comportamentul macroscopic
- Singura soluție: **observăm** sistemul – adică îl simulăm
- Același lucru pentru economii, ecosisteme, rețele sociale: cunoașterea regulilor individuale nu ajunge

E **Emergență**

Proprietăți ale întregului (autostrada, planorul, ambuteiajul) care nu apar în regula unei singure celule. Simularea este instrumentul prin care le descoperim.

Medii excitabile: modelul Greenberg–Hastings



stări: repaus (alb), excitat (roșu), refractar (galben → verde)

- Repaus → **excitat** dacă un vecin e excitat
- Excitat → **refractar** câțiva pași → repaus
- Apar **unde** și **spirale** care se autoîntrețin
- Modele pentru inimă (aritmii), neuroni, reacția Belousov–Zhabotinsky
- Aceeași logică S → I → R ca la epidemii, pe o grilă

Variante ale automatelor celulare

1 Stocastice

Regula se aplică cu o probabilitate: incendii, trafic cu încetiniri aleatoare (ora 2).

2 Asincrone

Celulele se actualizează pe rând: evită oscilațiile artificiale, dar pierd paralelismul (Cernazean).

3 Neuniforme

Reguli diferite în zone diferite: drumuri și intersecții, tipuri de teren.

4 Stări continue

Lenia (2019): stări reale, vecinătăți netede – „organisme” care se mișcă lin.

5 Neurale

Growing Neural CA (2020): o rețea neuronală mică învață regula care crește o imagine dată și o repară.

6 Pe rețele

Celule = noduri ale unui graf: modele de opinie, epidemii pe rețele de contacte (S9).

Automatele celulare și calculul paralel

- Fiecare celulă depinde doar de vecinii ei de la pasul anterior: toate celulele se pot calcula **simultan**
- Mașinile CAM (Toffoli și Margolus, MIT, anii 1980): hardware dedicat automatelor celulare
- Azi: **GPU**-uri cu mii de nuclee; fiecare fir de execuție calculează o celulă (sau un bloc)
- Pe mai multe calculatoare: grila se împarte în **blocuri**, care schimbă doar marginile (ora 3)
- Prețul actualizării asincrone: dependențe între celule, deci mult mai puțin paralelism

V

Von Neumann, din nou

Ironie istorică: von Neumann, care a dat numele arhitecturii secvențiale a calculatoarelor, a inventat și unul dintre cele mai paralele modele de calcul.

Verificare

? Din ce clasă Wolfram face parte **regula 30**, pornită dintr-o singură celulă?

A clasa I (punct fix)

B clasa II (periodic)

C clasa III (haotic)

D clasa IV (complex, universal)

De la jucării matematice la modele ale realității

- Cernazean: realitatea are mai multe niveluri – **microscopic**, **mezoscopic**, **macroscopic**
- Comportamentul macroscopic depinde **puțin** de detaliile microscopice: contează simetriile și **legile de conservare**
- Strategia: construim o lume fictivă, simplă și ușor de simulat în paralel, care are **același comportament macroscopic**
- Exemple în ora 2: mașini pe un inel → ambuteiaje reale; particule pe o rețea → curgerea fluidelor
- Un automat celular este o „caricatură” a realității – dar o caricatură care păstrează ce contează

F **Fulgul de zăpadă**

Cernazean: „Ce este aceasta?” – o regulă de o linie (o celulă îngheață dacă are exact un vecin înghețat) produce forme care seamănă izbitor cu fulgii reali.

Rezumatul orei 1

- ✓ Automat celular ⟨A, S, N, Φ⟩: rețea, stări, vecinătate (von Neumann, Moore), regulă locală, actualizare **sincronă**
- ✓ Von Neumann și Ulam: autoreproducere; regula parității copiază orice model
- ✓ **256** de reguli elementare; 30 (haos), 90 (Sierpiński), 110 (universală), 184 (trafic); clasele I–IV
- ✓ **Game of Life** (B3/S23): planoare, tunuri, calculator universal; NumPy de sute de ori mai rapid decât buclele
- ✓ Emergență și ireductibilitate (furnica lui Langton): uneori singura cale este să simulezi

Ora 2

Automatele celulare ca modele ale realității

1. Trafic: regula 184 și modelul Nagel–Schreckenberg
2. Incendii, percolare, fulgi de zăpadă, difuzie
3. Gaze pe rețea și metoda Lattice Boltzmann
4. Automate celulare în informatică: peșteri procedurale

Ce face un model CA să fie bun

- Alegem **nivelul** potrivit: o celulă poate fi o mașină, un copac, o moleculă – sau o populație de molecule
- Păstrăm **legile de conservare** ale fenomenului: mașinile nu dispar, masa și impulsul unui fluid se conservă
- Păstrăm **simetriile** importante: un fluid nu are direcții privilegiate, deci nici rețeaua nu trebuie să aibă
- Adăugăm **aleatorul** acolo unde detaliile microscopice nu contează sau nu le cunoaștem
- Validăm pe mărimi **macroscopice**: flux de trafic, viteză de propagare, profil de curgere

C **Criteriul lui Cernazean**

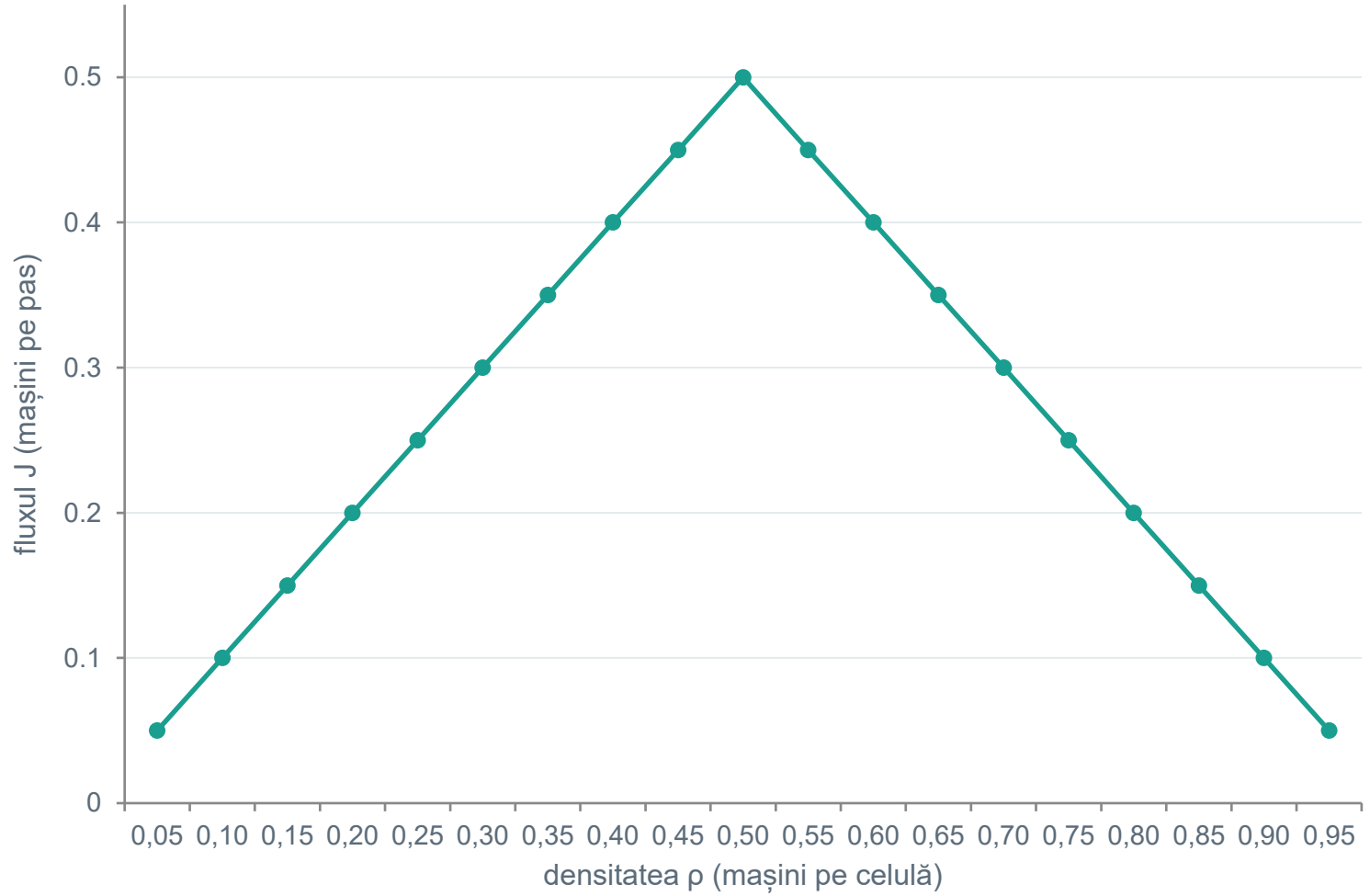
Un model CA bun este simplu, flexibil, intuitiv și eficient – și are comportamentul macroscopic corect, chiar dacă microscopic este o caricatură.

Trafic

Ambuteiaje din patru reguli

1. Regula 184: diagrama fundamentală
2. Modelul Nagel–Schreckenberg (1992)
3. Ambuteiaje fantomă și aplicații reale

Regula 184: fluxul de mașini în funcție de densitate



- Inel de 1 000 de celule, 500 de pași de rodaj
- Sub ρ = 0,5: toate mașinile se mișcă, **J = ρ**
- Peste ρ = 0,5: se mișcă doar mașinile din fața golurilor, **J = 1 – ρ**
- Flux maxim **0,5** la densitatea critică 0,5
- Forma de „cort” este **diagrama fundamentală** a traficului

Modelul Nagel–Schreckenberg (1992)

Viteze 0…v_{max} și patru reguli, aplicate tuturor mașinilor simultan

Drumul este un inel de celule (o celulă ≈ 7,5 m, un pas ≈ 1 s). Fiecare mașină are o viteză întreagă $v \in \{0, \ldots, v_{max}\}$, cu $v_{max} = 5$ (aproximativ 135 km/h).

Nagel și Schreckenberg, J. Phys. I France 2, 1992; Cernazean L4

- **1. Accelerare**: $v \leftarrow \min(v + 1, v_{max})$
- **2. Frânare**: $v \leftarrow \min(v, d)$, unde d = celulele libere până la mașina din față – fără accidente
- **3. Încetinire aleatoare**: cu probabilitatea p, $v \leftarrow \max(v - 1, 0)$ – șoferul neatent sau prudent
- **4. Deplasare**: fiecare mașină avansează v celule

Modelul Nagel–Schreckenberg în NumPy

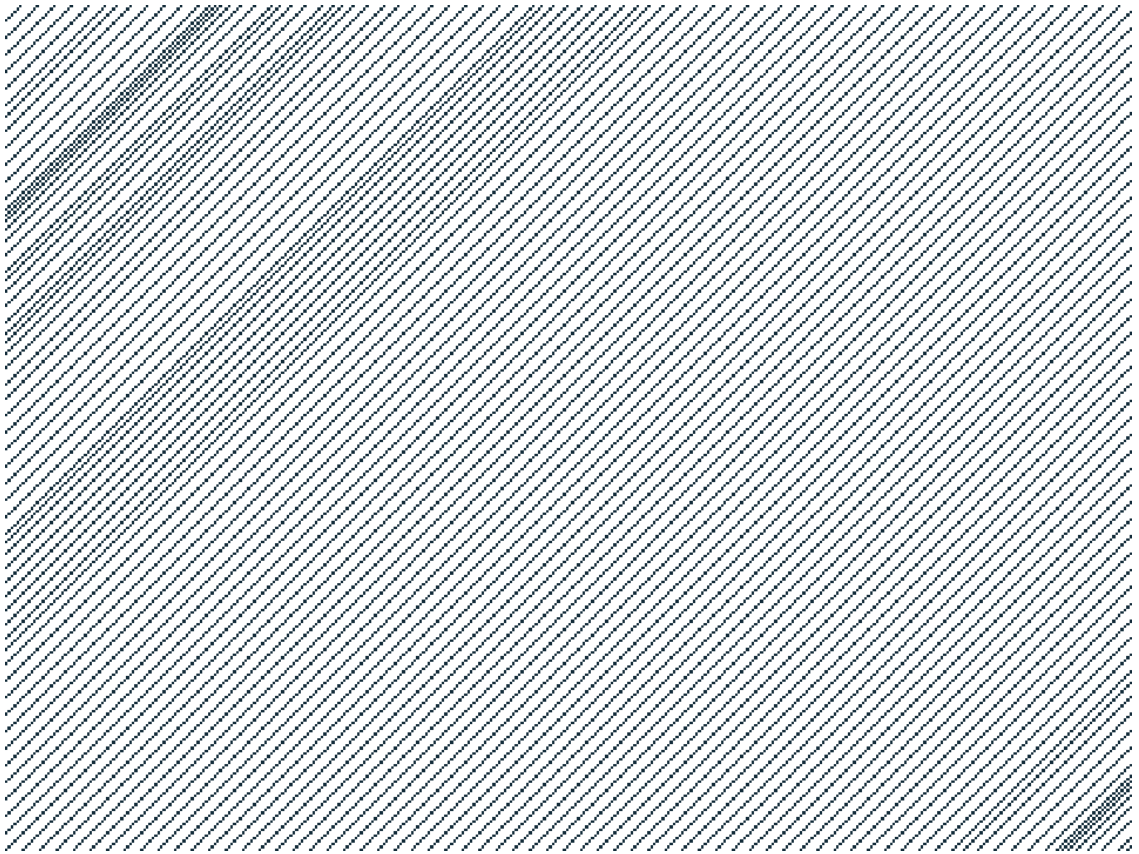
```
import numpy as np
rng = np.random.default_rng(86)
L, N, vmax, p = 1000, 150, 5, 0.25
pos = np.sort(rng.choice(L, N, replace=False))
v = np.zeros(N, int)

def nasch_step(pos, v):
    gap = (np.roll(pos, -1) - pos - 1) % L
    v = np.minimum(v + 1, vmax)       # 1. accelerare
    v = np.minimum(v, gap)            # 2. frânare
    slow = (rng.random(N) < p) & (v > 0)
    v = np.where(slow, v - 1, v)      # 3. aleator
    pos = (pos + v) % L               # 4. deplasare
    k = np.argsort(pos)
    return pos[k], v[k]

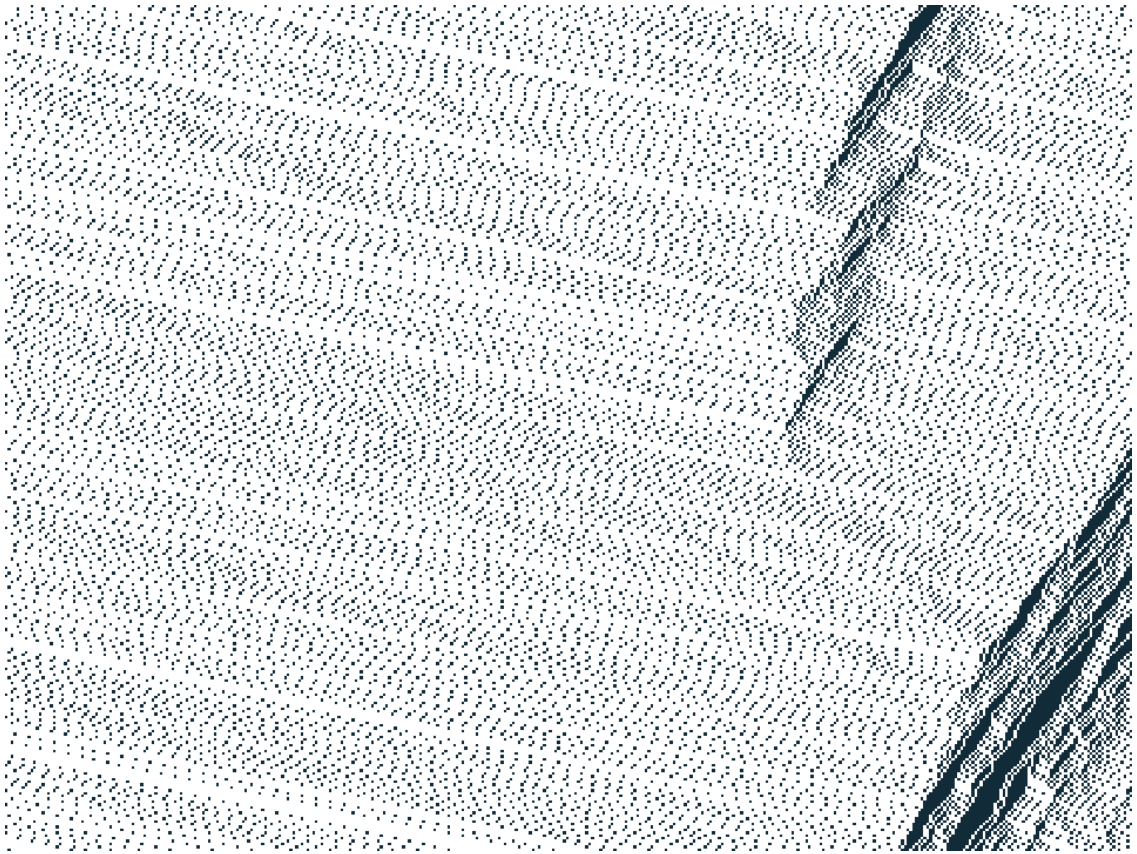
# fluxul: J = suma vitezelor / L
```

- Stocăm **mașinile**, nu celulele: N poziții și N viteze
- gap: distanța până la mașina din față, pe inel
- Cele 4 reguli = 4 operații vectoriale
- Sortarea păstrează ordinea mașinilor după trecerea prin capătul inelului
- Demo-ul săptămânii, partea a doua

Diagrame spațiu–timp: fără și cu încetiniri aleatoare (ρ = 0,18)

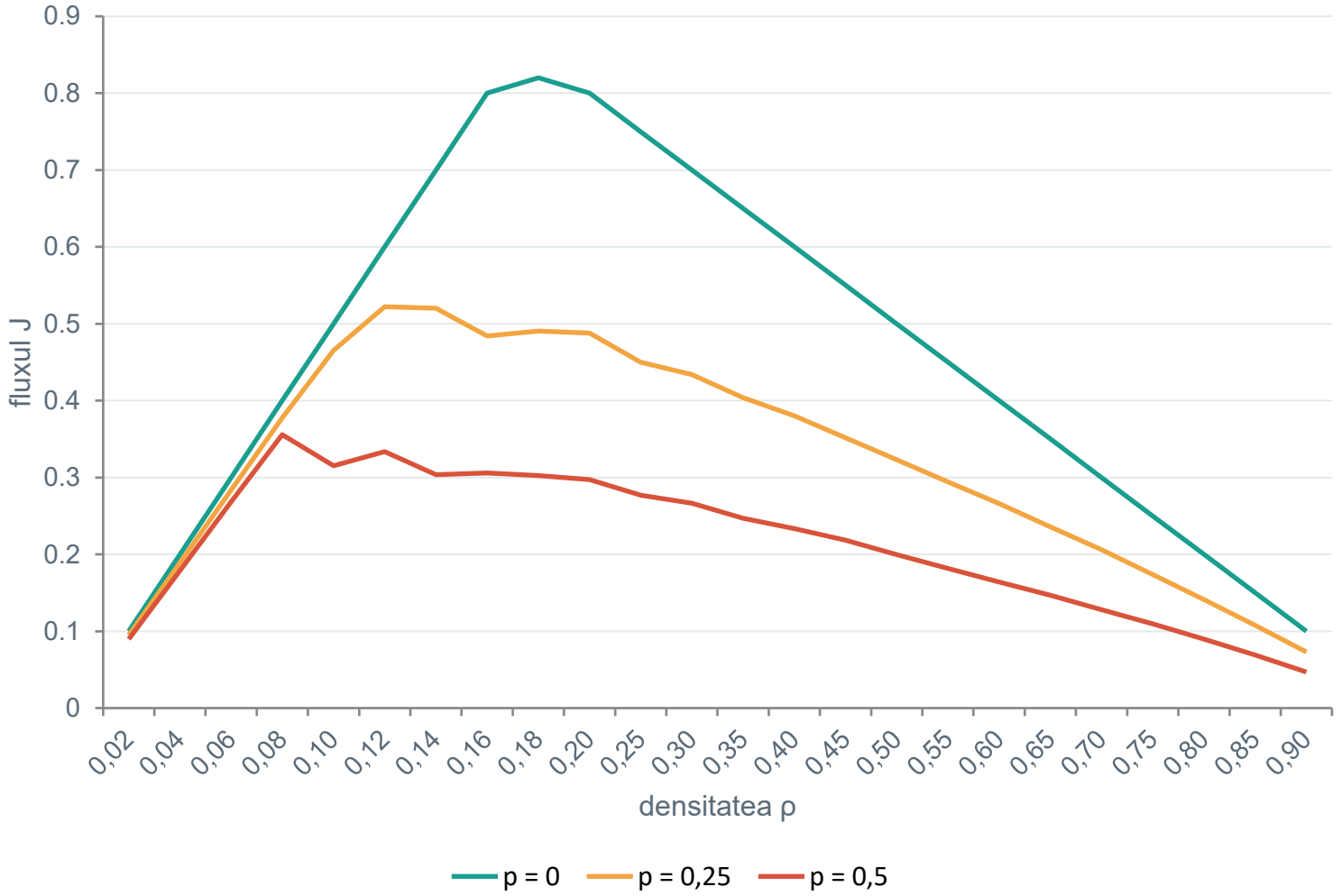


p = 0: după rodaj, trafic uniform, fără ambuteiaje



p = 0,25: ambuteiaje care apar spontan și se propagă **înapoi**

Diagrama fundamentală NaSch pentru trei valori ale lui p



- Inel de 1 000 de celule, $v_{max} = 5$, media ultimilor 500 de pași
- p = 0: maxim **0,82** la ρ ≈ 0,18
- p = 0,25: maxim **0,52**, la densitate mai mică
- Încetinirile aleatoare **reduc** capacitatea drumului
- Ramura din dreapta: trafic congestionat

Capacitatea drumului depinde de comportamentul șoferilor

Probabilitatea p	Flux maxim J	Densitatea critică ρc	Flux la ρ = 0,5
0,00	0,820	0,18	0,500
0,25	0,522	0,12	0,323
0,50	0,356	0,08	0,200

- La p = 0,25 drumul transportă cu circa **36%** mai puține mașini pe oră decât la p = 0
- Concluzie practică: **omogenizarea** vitezelor (limite variabile, pilot automat adaptiv) crește capacitatea
- Fluxul maxim, nu viteza maximă, este ce contează pentru un inginer de trafic

Ambuteiaje fantomă: experimentul Sugiyama (2008)

- **22 de mașini** pe un circuit circular de 230 m, la Nagoya; șoferii sunt rugați să mențină ~30 km/h
- Fără niciun obstacol, după câteva minute apare un **ambuteiaj** care se deplasează înapoi cu ~20 km/h
- Exact comportamentul din modelul NaSch cu p > 0: mici fluctuații **amplificate** de frânări
- Modelul a prezis fenomenul cu 16 ani **înaintea** experimentului controlat
- Studii ulterioare: o singură mașină autonomă bine controlată poate **amortiza** undele (Stern et al., 2018)

T **Ce explică modelul**

Ambuteiajele nu cer accidente sau lucrări: densitate peste cea critică + reacții întârziate sau aleatoare sunt suficiente.

Modele CA de trafic în practică

1 TRANSIMS

Los Alamos, anii 1990: micro-simulare CA pentru regiuni întregi (Dallas, Portland), cu milioane de călătorii.

2 OLSIM

Prognoza traficului pe autostrăzile din Renania de Nord-Westfalia, bazată pe modelul NaSch și pe date de la senzori.

3 Geneva (Cernazean)

Rețea urbană: 1 066 de intersecții, 3 145 de segmente, 560 886 de celule, 85 055 de mașini.

4 Pietoni

Modelul floor-field (Burstedde et al., 2001): evacuări din clădiri, stadioane, gări.

5 Intersecții

Sensuri giratorii vs semafoare, sincronizarea semafoarelor („unda verde”).

6 Mașini autonome

Ce se întâmplă cu fluxul când o fracțiune din mașini reacționează instantaneu și fără aleator?

Verificare

? Care regulă a modelului Nagel–Schreckenberg este responsabilă pentru apariția **spontană** a ambuteiajelor la densități moderate?

A accelerarea

B frânarea

C încetinirea aleatoare

D deplasarea

Mediu și materiale

Incendii, percolare, cristale, difuzie

1. Incendiul de pădure și pragul de percolare
2. Fulgul de zăpadă pe rețea hexagonală
3. Difuzia ca automat celular cu stări reale

Modelul incendiului de pădure

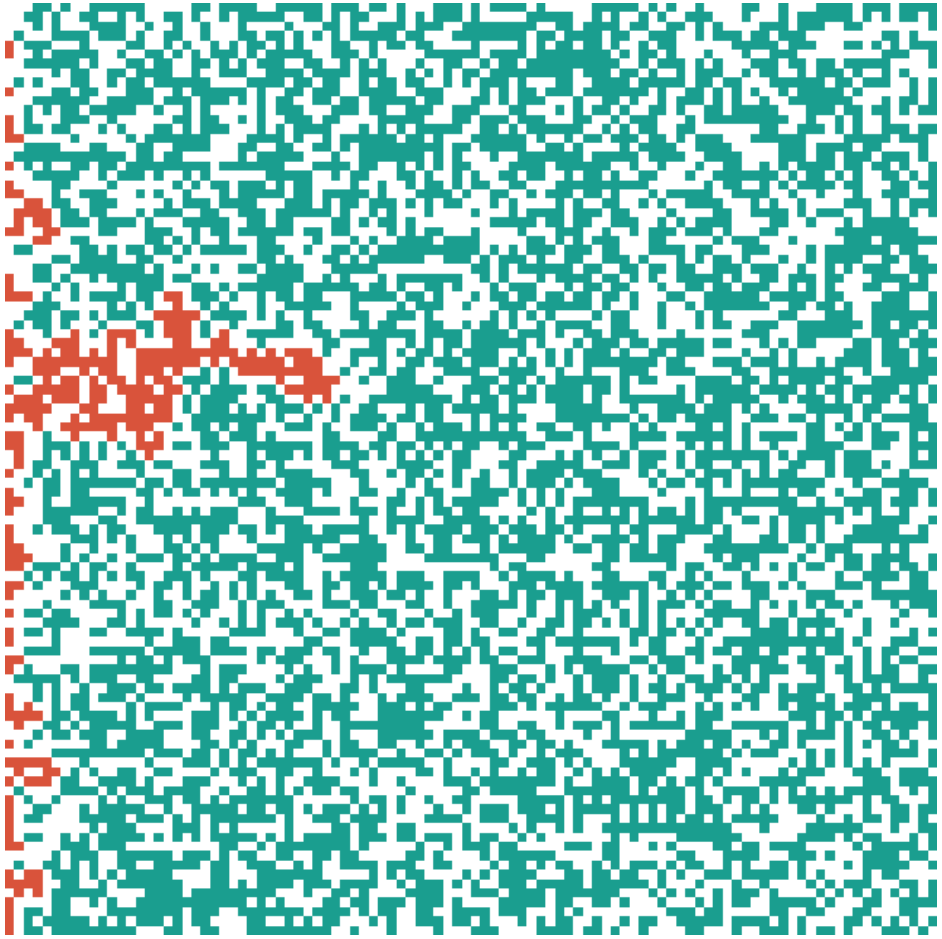
Stări: gol, copac, copac în flăcări, ars

O pădure pătrată în care fiecare celulă conține un copac cu probabilitatea p (**densitatea**). Focul pornește de pe marginea din stânga.

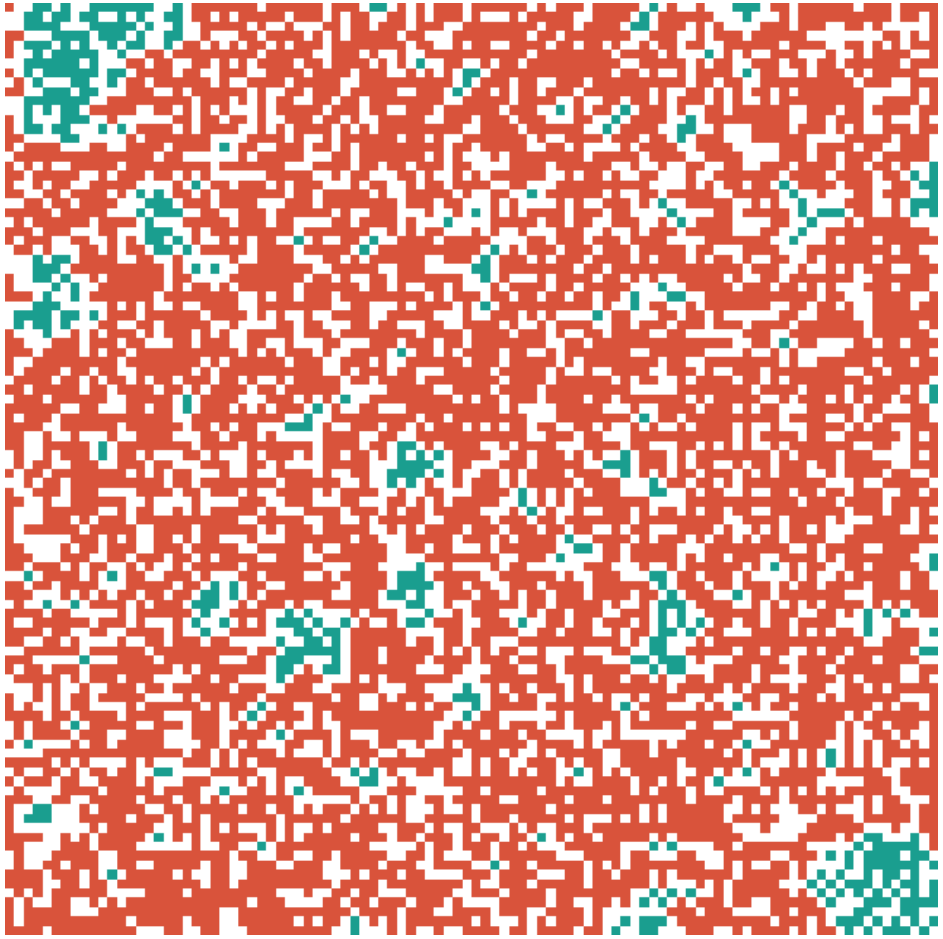
Cernazean L4; Drossel și Schwabl, 1992 (varianta cu creștere și fulgere)

- Un copac **arde** dacă un vecin (von Neumann) arde
- Un copac în flăcări devine **ars** la pasul următor
- Golurile nu ard niciodată
- Întrebarea: ce fracțiune din pădure arde și ajunge focul în partea dreaptă, în funcție de p?

Pădure de 100 × 100: o diferență mică de densitate, un rezultat foarte diferit

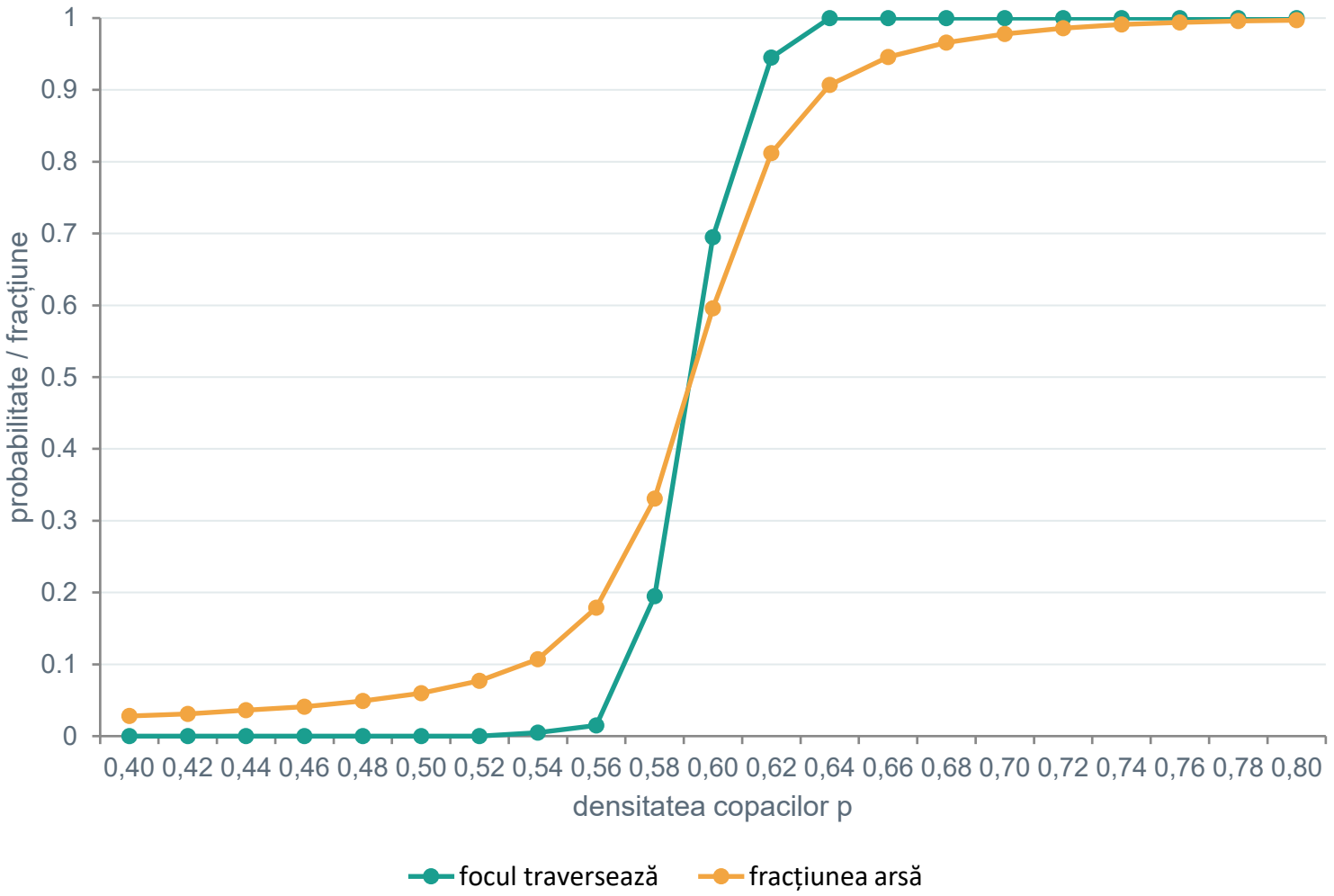


p = 0,55: arde 5,3% din copaci; focul se stinge



p = 0,65: arde 92,3% din copaci; focul traversează

Probabilitatea ca focul să traverseze pădurea



- 200 de păduri pentru fiecare p
- p = 0,58: traversează în **19,5%** din cazuri
- p = 0,62: în **94,5%**
- Pragul de percolare teoretic (rețea pătrată): **pc ≈ 0,5927**
- Cu cât pădurea e mai mare, cu atât saltul e mai abrupt

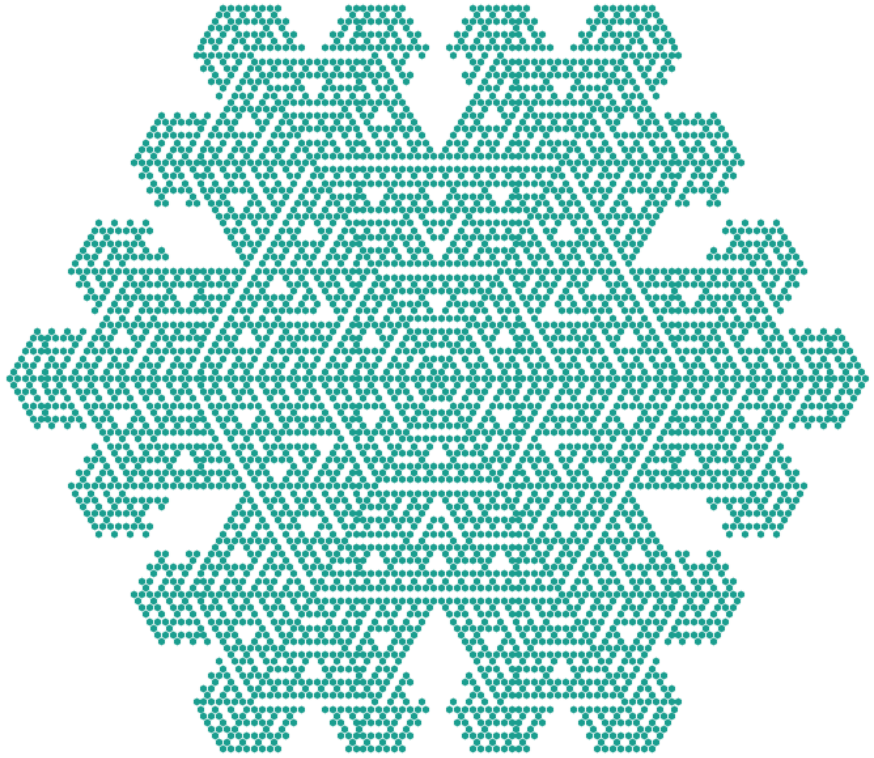
Percolarea: o tranziție de fază pe care o putem simula în câteva linii

- Sub pc: grupurile de copaci sunt **finite**; peste pc apare un grup **infinit** care traversează sistemul
- Aceeași matematică pentru: apa prin cafea sau roci poroase, curentul prin materiale compozite, epidemii pe rețele
- În jurul lui pc mărimile urmează **legi de putere** cu exponenți universali, care nu depind de detaliile rețelei
- Implementare eficientă: etichetarea componentelor conexe (scipy.ndimage.label) – nu e nevoie să simulăm pas cu pas
- Varianta dinamică (Drossel–Schwabl): copacii cresc, fulgerele aprind – **criticalitate autoorganizată**

P **Pentru politici publice**

Curățarea pădurii sub pragul critic (benzi fără vegetație) oprește incendiile mari; analog, vaccinarea sub pragul de imunitate oprește epidemiile.

Fulgul de zăpadă: o regulă de o linie pe o rețea hexagonală



55 de pași, 5 323 de celule înghețate

- Rețea **hexagonală**: 6 vecini, unghiuri de 60°
- Regula: o celulă **îngheață** dacă exact **un** vecin este înghețat
- Pornim de la o singură celulă
- Rezultat: simetrie de ordin 6 și ramificații, ca la fulgii reali (Packard, 1984)
- Modele mai realiste (Gravner–Griffeath, 2008) adaugă difuzia vaporilor

Biologie: diferențierea celulelor la Drosophila (Cernazean L4)

- În embrion, toate celulele unui strat sunt la început **identice**
- Apoi puțin sub **25%** devin celule nervoase (neuroblaste), restul celule ale epidermei
- Ipoteza biologică: fiecare celulă produce o substanță S; diferențierea are loc când S depășește un prag S_0
- **Inhibiție laterală**: celulele vecine își reduc reciproc producția de S
- Modelul CA: o regulă locală de „cine depășește primul pragul îi blochează pe vecini” reproduce proporția și distribuția observate

B **Același tipar, alte domenii**

Activare locală + inhibiție la distanță produce pete și dungi: blana animalelor, distribuția stomatelor pe frunze, modelele Turing.

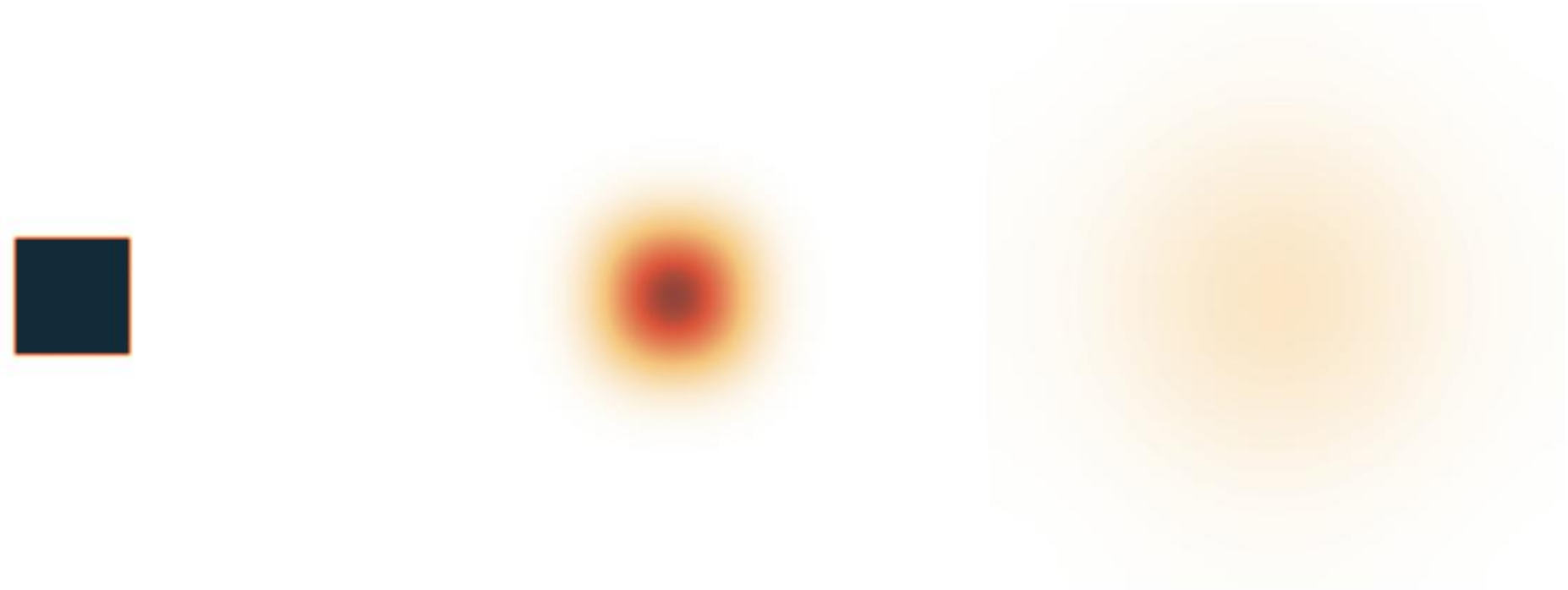
Difuzia ca automat celular cu stări reale

$$u(t+1) = u(t) + \alpha \cdot (\text{suma celor 4 vecini} - 4u)$$

regula	fiecare celulă schimbă substanță cu cei 4 vecini, proporțional cu **diferența** de concentrație
$\alpha = D\Delta t/\Delta x^2$	coeficientul de difuzie, adimensional; schema explicită este stabilă doar dacă **$\alpha \leq 1/4$**
conservare	ce pleacă dintr-o celulă ajunge în vecini: **masa totală** rămâne constantă

Aceasta este discretizarea explicită a ecuației căldurii $\partial u/\partial t = D\nabla^2 u$ – o EDP rezolvată ca automat celular. Legătura cu S7: pasul de timp prea mare face schema instabilă, exact ca la Euler pentru sisteme rigide.

Difuzia unui pătrat de substanță: pașii 0, 100 și 1 000



α = 0,2; grilă 100 × 100 cu margini periodice

Masa totală: **400** la început și **400** după 1 000 de pași – conservată exact (până la erorile de rotunjire). Cu α = 0,3 > 1/4, după doar 60 de pași valorile ajung la ±900 000: oscilații care cresc exponențial.

Fluide

De la gaze pe rețea la Lattice Boltzmann

1. Modelele HPP și FHP: particule booleene
2. Microdinamica: propagare și coliziune
3. Lattice Boltzmann: D2Q9, BGK, vâscozitate
4. Curgerea în jurul unui cilindru

Gaze pe rețea: HPP (1973) și FHP (1986)

HPP – rețea pătrată

- Hardy, de Pazzis, Pomeau
- 4 direcții, cel mult o particulă pe direcție
- Coliziune frontală: rotire cu 90°
- **Conservă** masa și impulsul
- Rețeaua pătrată e prea puțin simetrică: curgere **anizotropă**

vs

FHP – rețea hexagonală

- Frisch, Hasslacher, Pomeau
- 6 direcții, unghiuri de 60°
- Coliziuni cu 2 și 3 particule, unele aleatoare
- Conservă masa și impulsul
- Simetria hexagonală este suficientă: la scară mare apar ecuațiile **Navier–Stokes**

Cernazean (L4): „Conservarea exactă a masei și impulsului: asta contează cu adevărat pentru un fluid!” – plus o rețea suficient de simetrică.

Microdinamica unui gaz pe rețea

$$n_i(r + c_i, t + 1) = n_i(r, t) + \Omega_i(n(r, t))$$

$n_i \in \{0, 1\}$	există sau nu o particulă care se mișcă pe direcția i în nodul r
propagare	partea stângă: particula sare în nodul vecin r + c_i într-un pas de timp
Ω_i	operatorul de **coliziune**: rearanjează particulele dintr-un nod, conservând numărul și impulsul lor

Problema gazelor booleene: zgomot statistic mare (trebuie mediat pe zone mari) și artefacte ale rețelei. Soluția (McNamara și Zanetti, 1988): înlocuim $n_i \in \{0, 1\}$ cu media lui, $f_i \in [0; 1]$ – metoda Lattice Boltzmann.

Lattice Boltzmann cu coliziune BGK

$$f_i(x + c_i, t + 1) = f_i - \omega \cdot (f_i - f_i(eq))$$

f_i	densitatea de particule care se mișcă pe direcția i (număr real); $\rho = \Sigma f_i$, $\rho u = \Sigma f_i c_i$
$f_i(eq)$	echilibrul local: $f_i(eq) = w_i \rho(1 + 3c_i \cdot u + 4{,}5(c_i \cdot u)^2 - 1{,}5u^2)$
ω	rata de relaxare spre echilibru; fixează **vâscozitatea**: $\nu = (1/\omega - 1/2)/3$, deci $0 < \omega < 2$

Fiecare pas are două etape: coliziune (locală, în fiecare nod) și propagare (fiecare f_i se mută în vecinul de pe direcția i). Ambele sunt perfect paralele – de aceea LBM rulează excelent pe GPU.

Rețeaua D2Q9: 2 dimensiuni, 9 viteze

Direcții	Vectori c_i	Câte	Pondere w_i
repaus	(0, 0)	1	4/9
axe	(±1, 0), (0, ±1)	4	1/9
diagonale	(±1, ±1)	4	1/36

- Ponderile însumează 1: $4/9 + 4 \cdot 1/9 + 4 \cdot 1/36 = 1$
- Viteza sunetului pe rețea: $cs^2 = 1/3$; vâscozitatea: $\nu = cs^2 \cdot (1/\omega - 1/2)$
- Simularea noastră: Re = 150, $\nu = 0{,}00293$, deci $\omega = 1{,}9654$ – aproape de limita de stabilitate 2

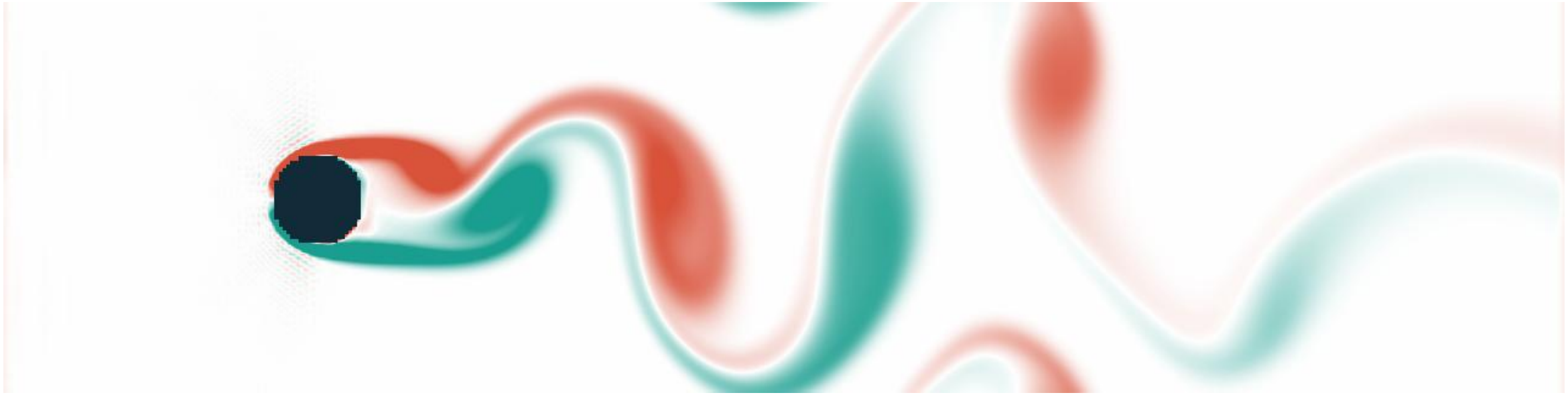
Nucleul unui solver LBM în NumPy

```
# f: (9, nx, ny); c: vectorii cᵢ; w: ponderile
def equilibrium(rho, u):
    cu = 3 * np.einsum('id,dxy->ixy', c, u)
    usq = 1.5 * (u[0]**2 + u[1]**2)
    return rho * w[:, None, None] * (
        1 + cu + 0.5 * cu**2 - usq)

for it in range(steps):
    rho = f.sum(axis=0)
    u = np.einsum('id,ixy->dxy', c, f) / rho
    feq = equilibrium(rho, u)
    fout = f - omega * (f - feq)      # coliziune
    for i in range(9):                # obstacol
        fout[i, obst] = f[8 - i, obst]
    for i in range(9):                # propagare
        f[i] = np.roll(fout[i], c[i], axis=(0, 1))
```

- Coliziune: o expresie pe toată grila
- **Bounce-back**: pe obstacol particulele se întorc pe direcția opusă (8 − i) – viteză nulă la perete
- Propagare: 9 deplasări de vectori, ca la Game of Life
- Lipsesc doar condițiile de intrare și ieșire (Zou–He) – în notebook

Curgere în jurul unui cilindru: aleea de vârtejuri von Kármán



vorticitatea (roșu: rotație într-un sens, verde-albastru: în celălalt); Re = 150, grilă 400 × 100

14 000 de pași LBM în ~108 s pe un singur nucleu, cu NumPy: aproximativ **5,2 milioane de actualizări de noduri pe secundă** (MLUPS). Un solver C++ optimizat atinge sute de MLUPS pe nucleu, iar un GPU modern mii.

Ce am obținut și de ce funcționează

- Niciun termen din ecuațiile **Navier–Stokes** nu apare explicit în cod – și totuși le rezolvăm
- Analiza Chapman–Enskog arată că, la scară mare, LBM reproduce Navier–Stokes cu vâscozitatea $\nu = (1/\omega - 1/2)/3$
- Condiții: viteze mici față de viteza sunetului pe rețea (aici $u = 0{,}04$, Mach $\approx 0{,}07$)
- Pereții complicați sunt ușori: bounce-back pe orice formă desenată pe grilă
- Prețul: grile mari pentru Re mare; la ω aproape de 2 schema devine instabilă – se folosesc operatori de coliziune mai avansați

R

Numărul Reynolds

$Re = u \cdot D / \nu$ compară inerția cu vâscozitatea. Peste $Re \approx 47$, curgerea în spatele unui cilindru devine periodică și apar vârtejurile alternante.

Lattice Boltzmann în practică

1 Palabos

Bibliotecă open-source C++ (Geneva), paralelizată transparent: domeniul este împărțit în blocuri (Cernazean L2, L5).

2 Aerodinamică

Simulări de mașini și avioane în industria auto și aerospațială; zgomotul aerodinamic.

3 Medicină

Curgerea sângelui în vase și anevrisme, din geometria obținută prin tomografie.

4 Medii poroase

Petrol, apă subterană, filtre: geometrii complicate tratate cu bounce-back.

5 Multifazic

Bule, picături, amestecuri de lichide cu tensiune superficială.

6 GPU

Operații locale și propagare regulată: unul dintre algoritmii cei mai potriviți pentru GPU.

Verificare

? Ce trebuie să **conserve** exact regulile de coliziune ale unui gaz pe rețea ca să obținem, la scară mare, un fluid?

A doar numărul de particule

B numărul de particule și impulsul

C energia cinetică a fiecărei particule

D poziția fiecărei particule

Automat celular sau ecuație cu derivate parțiale?

Automatul celular e potrivit când…

- fenomenul e **discret** (mașini, copaci, indivizi)
- știm regulile locale, nu și ecuația macroscopică
- geometria e complicată (pereți, pori)
- vrem paralelism masiv, simplu
- fluctuațiile și evenimentele rare contează

EDP-ul e potrivit când…

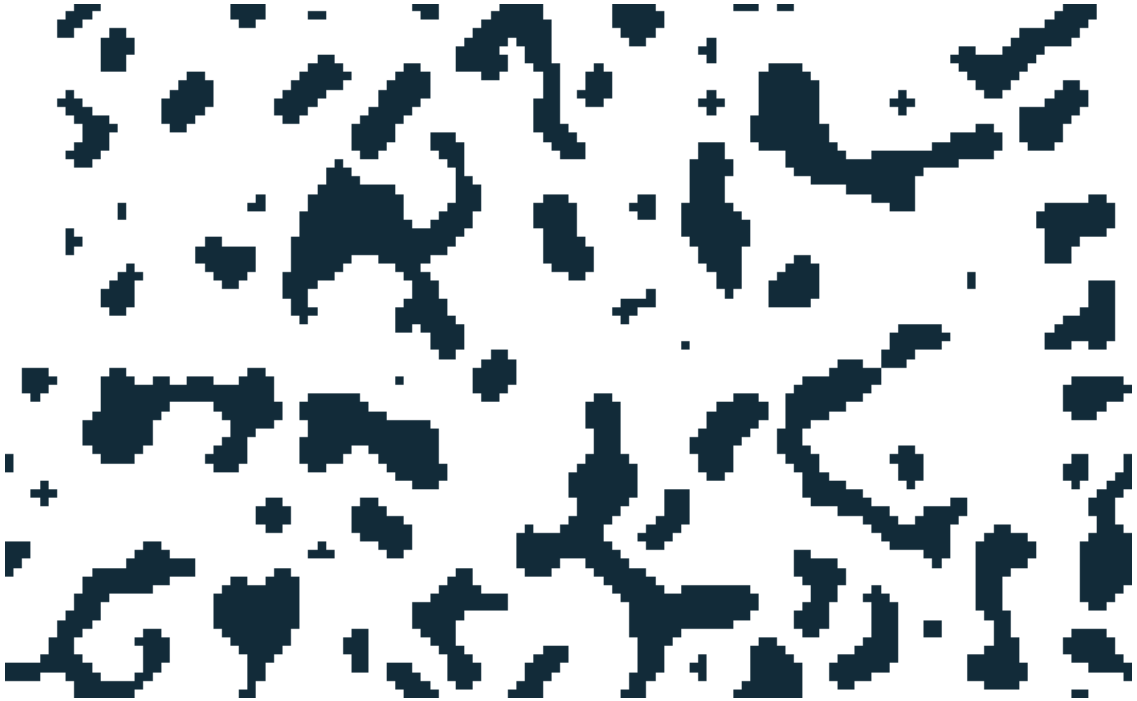
- avem o ecuație validată (căldură, elasticitate)
- avem nevoie de **precizie** controlată și analiză de erori
- mărimile sunt netede, fără granularitate
- există solvere mature (elemente finite)
- vrem să extragem formule, nu doar simulări

Granița este fluidă: difuzia explicită este simultan EDP discretizată și automat celular, iar LBM este folosit ca solver pentru Navier–Stokes.

În informatică: generarea procedurală a peșterilor în jocuri



start: 45% pereți, aleator



după 5 pași cu regula majorității: peșteri organice

Alte aplicații în informatică

- **Procesarea imaginilor**: eroziune, dilatare, filtrul majorității pentru eliminarea zgomotului – reguli CA pe pixeli
- **Numere aleatoare și criptografie**: regula 30 și automate hibride 30/150 în generatoare hardware
- **Wireworld**: un automat cu 4 stări în care se construiesc fire, diode, porți logice – chiar un calculator
- **Shadere GPU**: simulări de apă, nisip, foc în jocuri (de ex. Noita: fiecare pixel este o celulă)
- **Calcul neconvențional**: automate celulare cuantice cu puncte (QCA) ca alternativă la tranzistoare

P **Pentru proiecte**

Un automat celular este un proiect excelent de semestru: o regulă, o grilă, un mod de vizualizare – și o întrebare la care să răspundă simularea.

Cum construim un model cu automate celulare

1

Fenomenul

Ce mărime macroscopică vrem să explicăm sau să prezicem?

2

Celula

Ce reprezintă o celulă, ce stări are, ce vecinătate?

3

Regulile

Ce se conservă? Ce e aleator? Sincron sau asincron?

4

Calibrarea

Parametrii (p, v_{max}, ω) din date sau din fizică

5

Validarea

Comparăm mărimi macroscopice: diagrama fundamentală, pragul, profilul

Limitele automatelor celulare

- **Anizotropia rețelei**: pe o grilă pătrată, petele cresc ca romburi sau pătrate, nu ca cercuri – vecinătăți mai mari sau aleator ajută
- **Artefactele actualizării sincrone**: oscilații care nu există în realitate (de aceea unele modele folosesc actualizare asincronă)
- **Calibrarea**: regulile simple au puțini parametri, dar legătura lor cu mărimile fizice nu e mereu evidentă
- **Rezoluția**: o celulă = o mașină e firesc; o celulă = un metru pătrat de pădure e o alegere care trebuie justificată
- **Memoria**: grile 3D mari ocupă repede zeci de GB (LBM D3Q19: 19 numere pe nod)

Verificare

? Rulăm regula de difuzie explicită cu **α = 0,3** pe o grilă 2D. Ce observăm?

A difuzie mai rapidă, dar corectă

B masa totală scade treptat

C oscilații care cresc exponențial (instabilitate)

D substanța rămâne pe loc

Modelele orei 2 dintr-o privire

Model	Stări	Regula esențială	Ce validăm
Regula 184	mașină / gol	înaintează dacă e liber în față	diagrama fundamentală
NaSch	viteză 0…5	accelerare, frânare, aleator, deplasare	ambuteiaje, capacitate
Incendiu	gol, copac, foc, ars	arde dacă un vecin arde	pragul pc ≈ 0,593
Fulg	vapori / gheață	îngheață cu exact 1 vecin înghețat	forma, simetria
Difuzie	concentrație reală	schimb cu vecinii, $\alpha \leq 1/4$	conservarea masei
LBM D2Q9	9 densități f_i	coliziune BGK + propagare	vârtejuri, Re

Rezumatul orei 2

- ✓ Un model CA bun păstrează **conservarea** și **simetriile**; validarea se face pe mărimi macroscopice
- ✓ Trafic: regula 184 și **Nagel–Schreckenberg** – încetinirea aleatoare produce ambuteiaje fantomă și reduce capacitatea
- ✓ Incendiu și **percolare**: tranziție bruscă la pc ≈ 0,593; fulgul de zăpadă din regula „exact un vecin”
- ✓ Difuzia = EDP discretizată = CA cu stări reale; stabilă doar pentru $\alpha \leq 1/4$
- ✓ Gaze pe rețea (HPP, FHP) și **Lattice Boltzmann** D2Q9: Navier–Stokes din coliziune și propagare

Ora 3

Particule și simulare de înaltă performanță

1. Legile lui Newton, potențiale, integrarea Verlet
2. Costul $O(N^2)$: raza de tăiere, grila, Barnes–Hut
3. Performanță: bucle, NumPy, C++, paralelism, GPU
4. Legea lui Amdahl și supercalculatoarele exascale

Două moduri de a descrie spațiul

Pe grilă (euleriană)

- Urmărim ce se întâmplă în **puncte fixe** ale spațiului
- Automate celulare, LBM, ecuații cu derivate parțiale
- Cost proporțional cu numărul de celule – și cu spațiul gol
- Natural pentru câmpuri: temperatură, concentrație, viteză

vs

Cu particule (lagrangiană)

- Urmărim **obiectele** care se mișcă: poziție, viteză, masă
- Stele, molecule, celule de sânge, boabe de nisip, personaje
- Cost proporțional cu numărul de particule – spațiul gol e gratuit
- Natural pentru obiecte distincte și deformări mari

Cernazean (L6): obiectele sunt modelate ca **puncte** – poziție, viteză și câteva proprietăți simple – ca să putem simula foarte multe; de exemplu ~1 miliard de globule roșii într-o arteră.

Unde apar sistemele de particule

1 Astrofizică

Problema celor N corpuri: galaxii care se ciocnesc, formarea structurilor cosmice cu miliarde de particule.

2 Dinamică moleculară

Proteine, medicamente, materiale: atomi care interacționează prin potențiale empirice (GROMACS, LAMMPS).

3 Medicină

Globule roșii purtate de sânge: interacțiune particulă–particulă plus cuplaj cu fluidul (Cernazean L6).

4 Materiale granulare

Nisip, cereale în silozuri, pastile pe benzi: metoda elementelor discrete (DEM).

5 Fluide fără grilă

SPH: fluidul este un nor de particule – valuri, explozii, efecte vizuale în filme.

6 Jocuri și grafică

Motoare fizice (Box2D, PhysX, Havok): corpuri rigide, haine, fum, scântei, la 60 de cadre pe secundă.

Ecuațiile de mișcare: legea a doua a lui Newton

$$m_i \cdot d^2 r_i / dt^2 = F_i = \Sigma_j \, F(r_i, r_j)$$

forțe pe perechi	forța totală pe particula i este suma interacțiunilor cu toate celelalte particule j
$\boldsymbol{F = -\nabla V}$	forța derivă dintr-un **potențial** V(r) care depinde doar de distanța r dintre particule
legea a treia	$F_{ij} = -F_{ji}$: fiecare pereche se calculează **o singură dată**, deci N(N − 1)/2 perechi

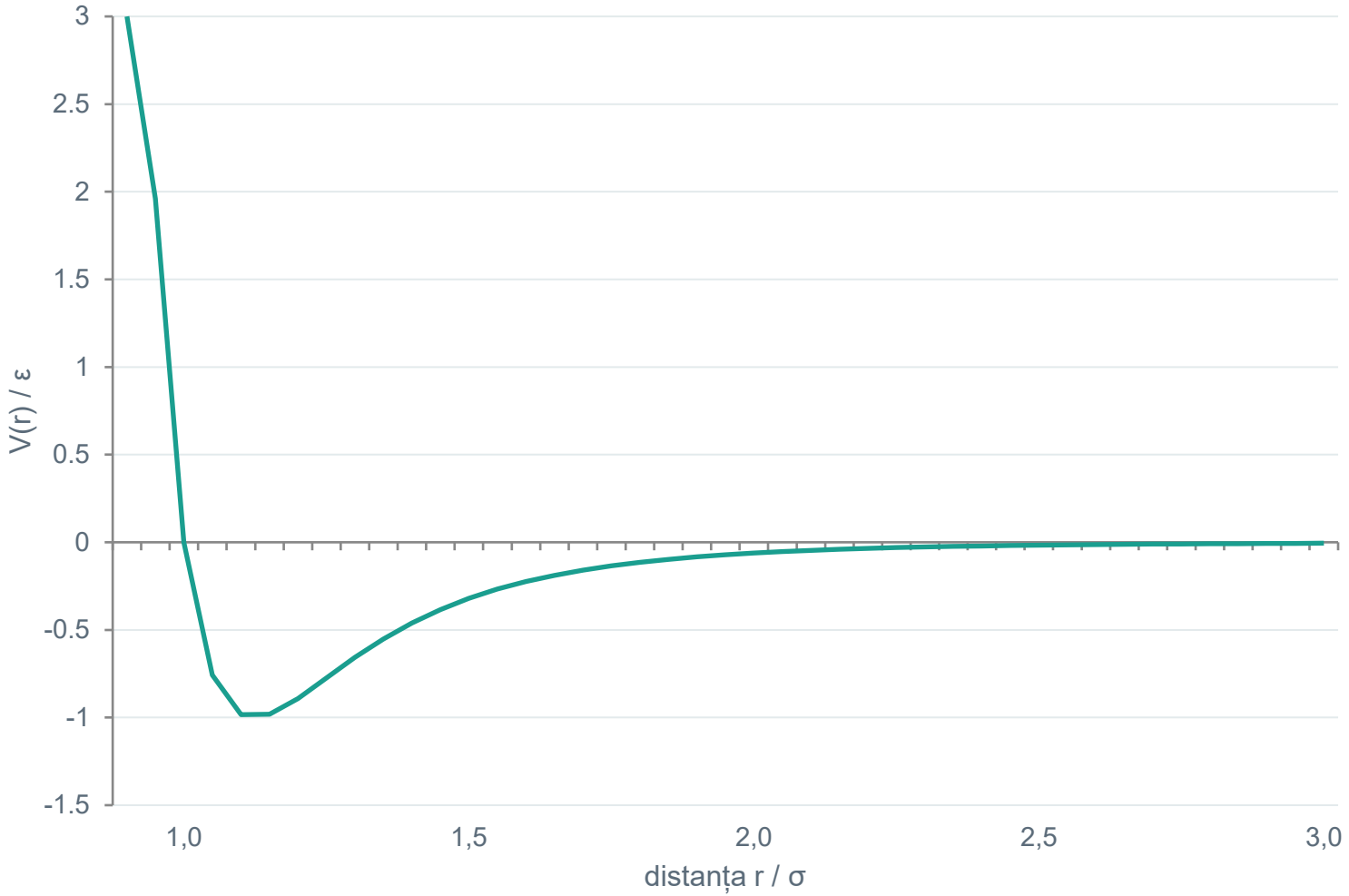
Pentru N particule în 2D sau 3D avem un sistem de 2N sau 3N ecuații diferențiale de ordinul 2 – exact tipul de sistem din S7, doar mult mai mare. Dificultatea nu este integrarea, ci calculul forțelor.

Potențiale de interacțiune frecvente

Potențial	V(r)	Rază de acțiune	Folosit pentru
Gravitație	$-G \cdot m_1 m_2 / r$	lungă: fără tăiere	stele, galaxii, planete
Coulomb	$k \cdot q_1 q_2 / r$	lungă: fără tăiere	ioni, plasme, proteine
Lennard-Jones	$4\varepsilon[(\sigma/r)^{12} - (\sigma/r)^{6}]$	scurtă: tăiere la $2{,}5\sigma$	gaze nobile, lichide simple
Resort (Hooke)	$k(r - r_0)^2 / 2$	doar perechi legate	legături chimice, haine, frânghii
Contact (Hertz)	proporțional cu $\delta^{5/2}$	doar la atingere ($\delta > 0$)	nisip, bile, materiale granulare

- Potențialele cu rază **lungă** cer tratarea tuturor perechilor (sau Barnes–Hut); cele cu rază **scurtă** permit raza de tăiere și grila
- Cernazean (L6): interacțiunile non-clasice (cuantice) se aproximează cu forțe de tip newtonian

Potențialul Lennard-Jones (σ = ε = 1)



- **Respingere** puternică la distanțe mici (termenul r^{-12}: norii de electroni)
- **Atracție** slabă la distanțe medii (r^{-6}: forțe van der Waals)
- Minimul: $V = -\varepsilon$ la $r = 2^{1/6}\sigma \approx$ **1,1225σ**
- La 2,5σ: V = -0,0163 – doar **1,6%** din adâncimea ε, deci tăierea e justificată

Integrare

Metoda Verlet



1. Verlet pentru poziții (1967)
2. Velocity Verlet și leapfrog
3. Orbita Kepler: Euler vs RK4 vs Verlet
4. De ce contează simplecticitatea

Algoritmul Verlet original (1967)

$$x(t + \Delta t) = 2x(t) - x(t - \Delta t) + a(t) \cdot \Delta t^2$$

derivare	adunăm seriile Taylor pentru x(t + Δt) și x(t − Δt): termenii cu v și cu Δt^3 se anulează
precizie	eroare locală $O(\Delta t^4)$ pentru poziții, cu **o singură** evaluare a forțelor pe pas
viteza	nu apare în formulă; se poate estima ulterior: v(t) ≈ [x(t + Δt) − x(t − Δt)] / 2Δt

Loup Verlet a folosit metoda în 1967 pentru simularea argonului cu potențial Lennard-Jones. Dezavantaj: avem nevoie de două poziții anterioare la start, iar vitezele (deci energia cinetică) sunt mai puțin precise.

Velocity Verlet: aceeași metodă, scrisă cu viteze

$$v := v + a\cdot\Delta t/2; \quad x := x + v\cdot\Delta t; \quad v := v + a(x)\cdot\Delta t/2$$

1. jumătate de impuls	actualizăm viteza cu accelerația veche, pe jumătate de pas
2. deplasare	mutăm particulele un pas întreg cu viteza de la mijlocul pasului
3. jumătate de impuls	calculăm forțele în pozițiile noi (singura evaluare) și terminăm actualizarea vitezei

Varianta leapfrog („săritura broaștei”, Cernazean L6) păstrează vitezele la jumătăți de pas: $v(t + \Delta t/2) = v(t - \Delta t/2) + a(t)\Delta t$, apoi $x(t + \Delta t) = x(t) + v(t + \Delta t/2)\Delta t$. Toate trei variantele produc aceleași traiectorii.

Velocity Verlet în Python

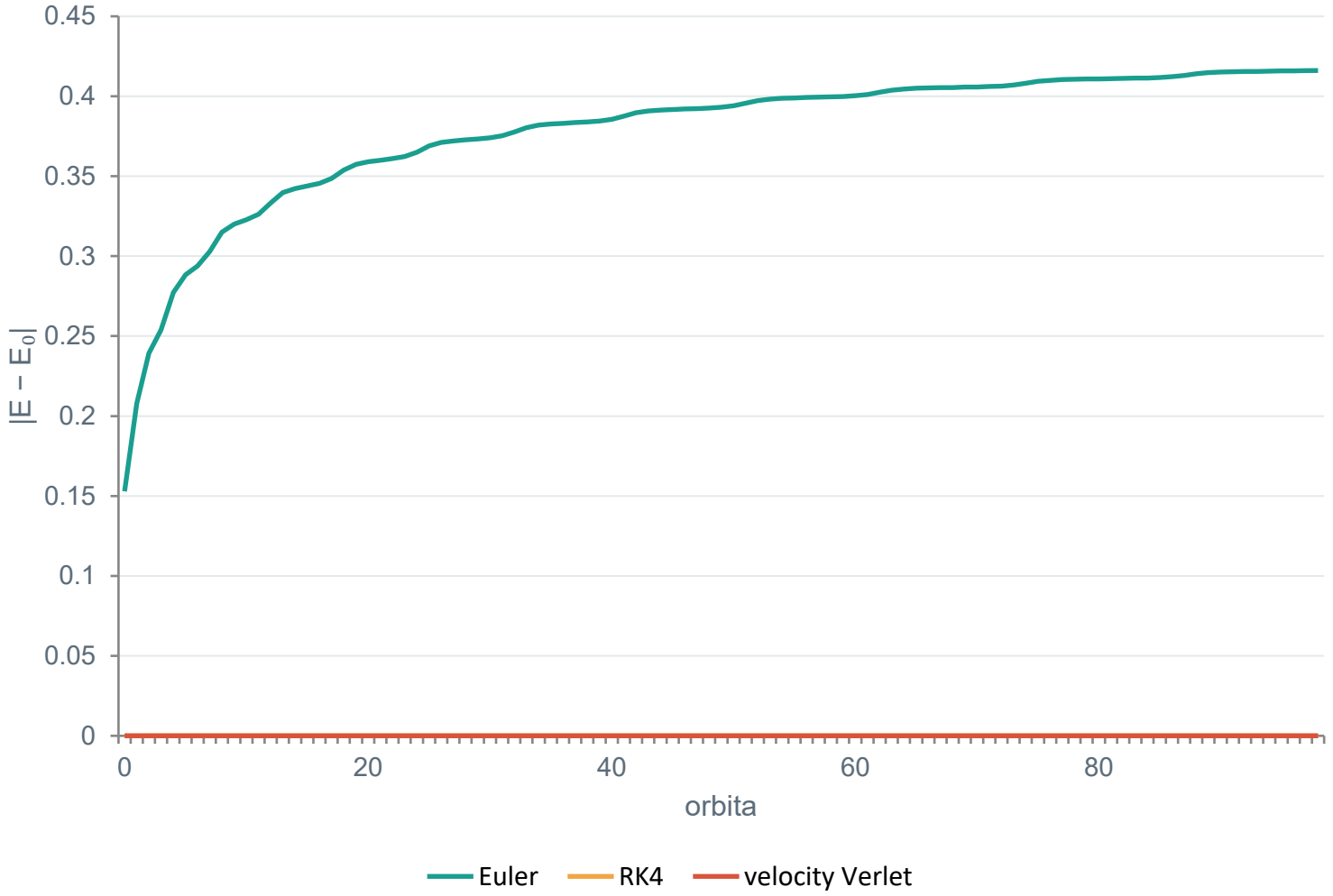
```
import numpy as np

def velocity_verlet(x, v, acc, dt, steps):
    a = acc(x)
    for _ in range(steps):
        v_half = v + 0.5 * dt * a    # 1. jumătate
        x = x + dt * v_half          # 2. deplasare
        a = acc(x)                   # O SINGURĂ evaluare
        v = v_half + 0.5 * dt * a    # 3. jumătate
    return x, v

# planetă în jurul Soarelui: GM = 1, orbită r = 1
acc = lambda p: -p / np.linalg.norm(p)**3
x, v = np.array([1.0, 0.0]), np.array([0.0, 1.0])
x, v = velocity_verlet(x, v, acc, 0.05, 12566)
# 12 566 de pași = 100 de orbite
```

- Funcționează pentru **orice** acc(x): gravitație, LJ, resorturi
- x și v pot fi vectori de N × 3 – toate particulele odată
- Același cost pe pas ca Euler
- Energia totală: $E = v^2/2 - 1/r$, exact −0,5 pentru orbita circulară

Eroarea maximă a energiei pe fiecare orbită (Δt = 0,05)



- **Euler**: energia crește cu 0,416 – de la −0,5 la -0,084; planeta aproape evadează
- **RK4**: $2{,}71 \cdot 10^{-6}$ la orbita 100 – invizibil aici, dar crește liniar
- **Verlet**: $7{,}78 \cdot 10^{-7}$ la **fiecare** orbită – mărginită, nu crește
- RK4 și Verlet se suprapun pe axa zero la această scară

Euler, RK4 și Verlet pe aceeași orbită

Metoda	Forțe pe pas	Eroarea maximă pe orbita 100	Comportamentul energiei
Euler	1	+0,416	crește sistematic: orbita se lărgește
RK4	4	$2{,}71 \cdot 10^{-6}$	mică, dar **derivă** (crește liniar în timp)
velocity Verlet	1	$7{,}78 \cdot 10^{-7}$	**oscilează**, mărginită: nu se acumulează

- Deja la orbita 100, Verlet are eroare **mai mică** decât RK4, deși costă de **4 ori** mai puțin; la 1 000 de orbite diferența crește de 10 ori
- În dinamica moleculară se fac 10^6–10^9 pași: o derivă oricât de mică devine vizibilă, o eroare mărginită nu
- Legătura cu S7: Euler simplectic păstra orbitele Lotka–Volterra – Verlet este varianta lui de ordinul 2

De ce Verlet este standardul în dinamica moleculară

- **Simplectică**: păstrează volumul în spațiul fazelor; conservă exact o energie „vecină”, deci eroarea energiei rămâne mărginită
- **Reversibilă în timp**: rulat cu $-\Delta t$, revine exact în starea inițială (până la rotunjire) – ca legile fizicii
- **Ieftină**: o evaluare a forțelor pe pas – iar forțele sunt partea scumpă
- **Memorie mică**: poziții, viteze, accelerații – nimic în plus
- Limitări: pas fix; pentru coliziuni violente sau stele foarte apropiate se folosesc pași adaptivi sau individuali

S **Ideea de fond**

Pentru simulări lungi, contează mai mult să păstrezi **structura** sistemului (conservarea energiei, reversibilitatea) decât precizia unui singur pas.

Verificare

? Simulăm o proteină timp de **10^8 pași**. De ce alegem velocity Verlet în locul lui RK4?

A Verlet are ordin de precizie mai mare decât RK4

B Verlet: o evaluare de forțe pe pas și eroare de energie mărginită

C RK4 nu poate integra ecuații de ordinul 2

D Verlet folosește pas adaptiv

Complexitate

Costul $O(N^2)$ și cum scăpăm de el

1. Raza de tăiere și metoda grilei: O(N)
2. Barnes–Hut: O(N log N) pentru gravitație
3. Ce măsurăm în practică

Raza de tăiere și metoda grilei (cell list)

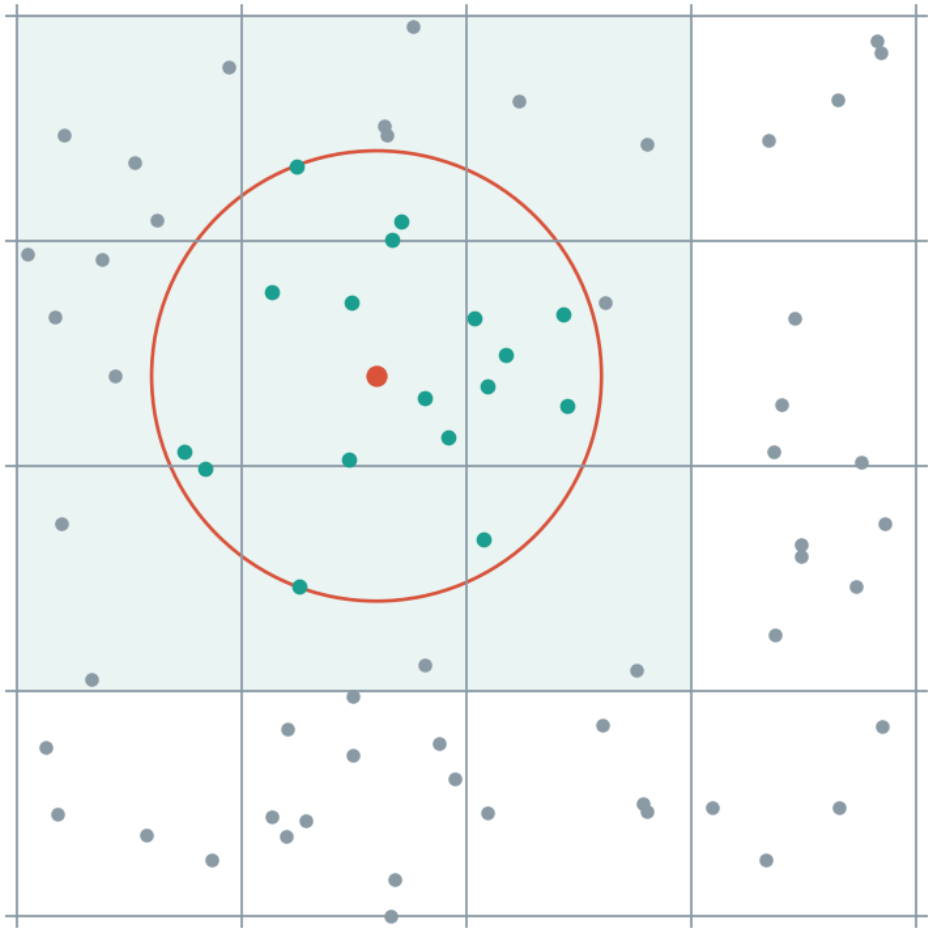
Interacțiunile cu rază scurtă cer doar vecinii apropiați

Cu un milion de particule avem ~$5 \cdot 10^{11}$ perechi pe pas: la ~10^9 perechi pe secundă, **8 minute pentru un singur pas** (Cernazean: „putem calcula N^2 interacțiuni într-o secundă?”). Cu raza de tăiere r_t, fiecare particulă are doar câteva zeci de vecini.

Cernazean L6: „Grid method”; Allen și Tildesley, Computer Simulation of Liquids

- Împărțim spațiul în celule de latură ≥ r_t
- Fiecare particulă se înscrie în celula ei: O(N), la fiecare pas
- Vecinii unei particule pot fi doar în celula ei și în cele **8** din jur (26 în 3D)
- Cost total: **O(N)** – la densitate constantă, fiecare particulă are un număr constant de vecini

Metoda grilei: căutăm vecinii doar în 9 celule



particula roșie, cercul de rază r_t și cele 9 celule examinate (zona colorată)

- Latura celulei = r_t: cercul încape mereu în cele 9 celule
- Dintre particulele examinate, doar cele din cerc (verzi) interacționează
- În loc de N − 1 distanțe, calculăm câteva zeci
- Grila se reconstruiește la fiecare pas (sau la câțiva pași, cu liste Verlet de vecini)

Metoda grilei în Python și varianta „de bibliotecă”

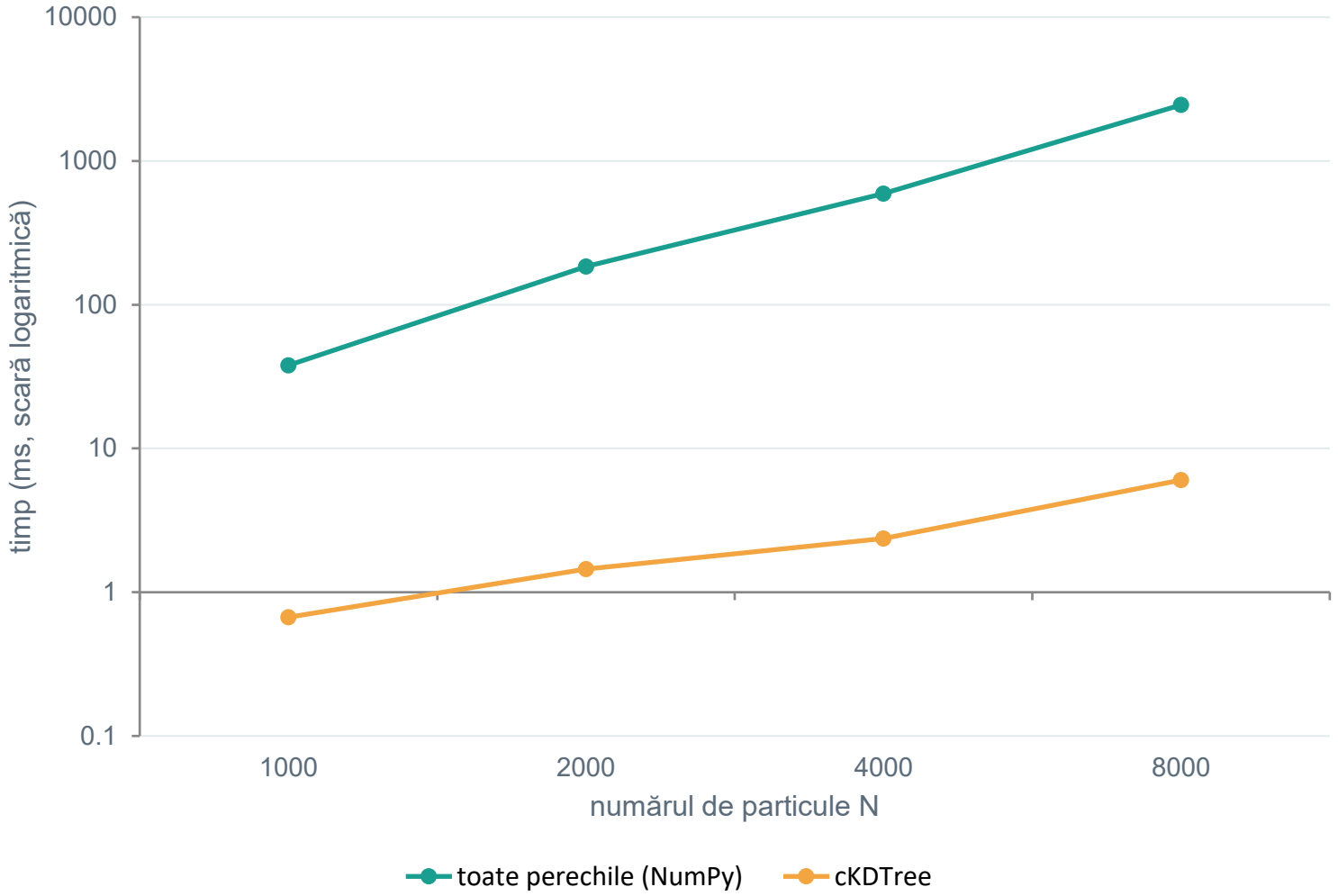
```
rc = 2.5
cell = np.floor(P / rc).astype(int)      # celula fiecăruia
grid = {}
for i, c in enumerate(map(tuple, cell)):
    grid.setdefault(c, []).append(i)

def neighbours(i):
    cx, cy = cell[i]
    for dx in (-1, 0, 1):
        for dy in (-1, 0, 1):
            for j in grid.get((cx + dx, cy + dy), []):
                d = P[j] - P[i]
                if j != i and d @ d < rc * rc:
                    yield j

# aceeași idee, optimizată (arbore k-d în C):
from scipy.spatial import cKDTree
pairs = cKDTree(P).query_pairs(rc, output_type='ndarray')
```

- Dicționar: celulă → lista particulelor
- Comparăm pătratele distanțelor: fără radical
- În producție: arbori k-d (SciPy) sau grile pe GPU
- Măsurătorile de pe slide-ul următor folosesc cKDTree

Măsurat: toate perechile vs arbore k-d (densitate 0,5, $r_t = 2,5$)



- N = 8 000: **2 453 ms** vs **6,03 ms** – de ~407 ori mai rapid
- Dublăm N: perechile **×4**, arborele **×2**
- N = 16 000: arborele găsește 77 466 de perechi în 11,1 ms
- Ambele metode găsesc **exact** aceleași perechi (verificat)

Gravitația nu are rază de tăiere: algoritmul Barnes–Hut (1986)

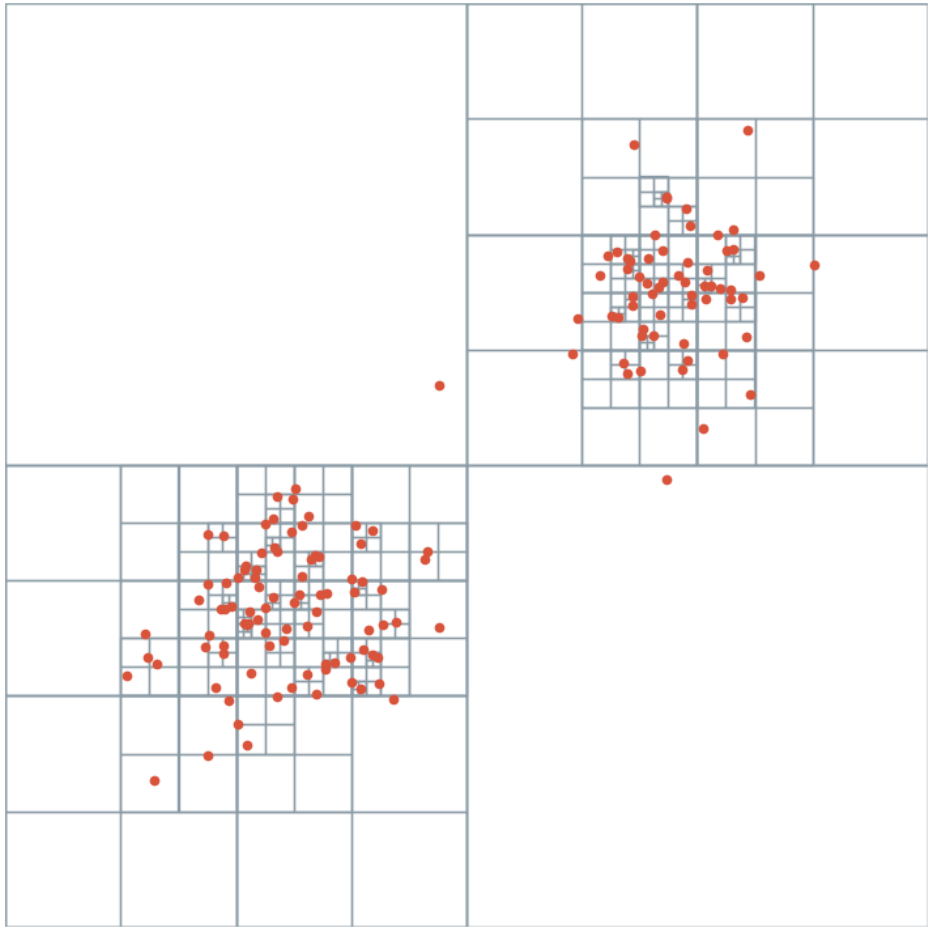
Un grup de corpuri îndepărtate se comportă ca un singur corp

Spațiul se împarte recursiv în pătrate (**quadtree**; octree în 3D) până când fiecare celulă are cel mult un corp. Fiecare nod reține masa totală și **centrul de masă** al corpurilor din el.

Barnes și Hut, Nature 324, 1986; Cernazean L6

- Pentru forța pe un corp, parcurgem arborele de la rădăcină
- Dacă s / d < **θ** (s = latura celulei, d = distanța până la centrul ei de masă), tratăm toată celula ca un singur corp
- Altfel coborâm în cei 4 fii
- θ = 0: exact ($O(N^2)$); θ ≈ 0,5: precizie ~1%, cost **O(N log N)**

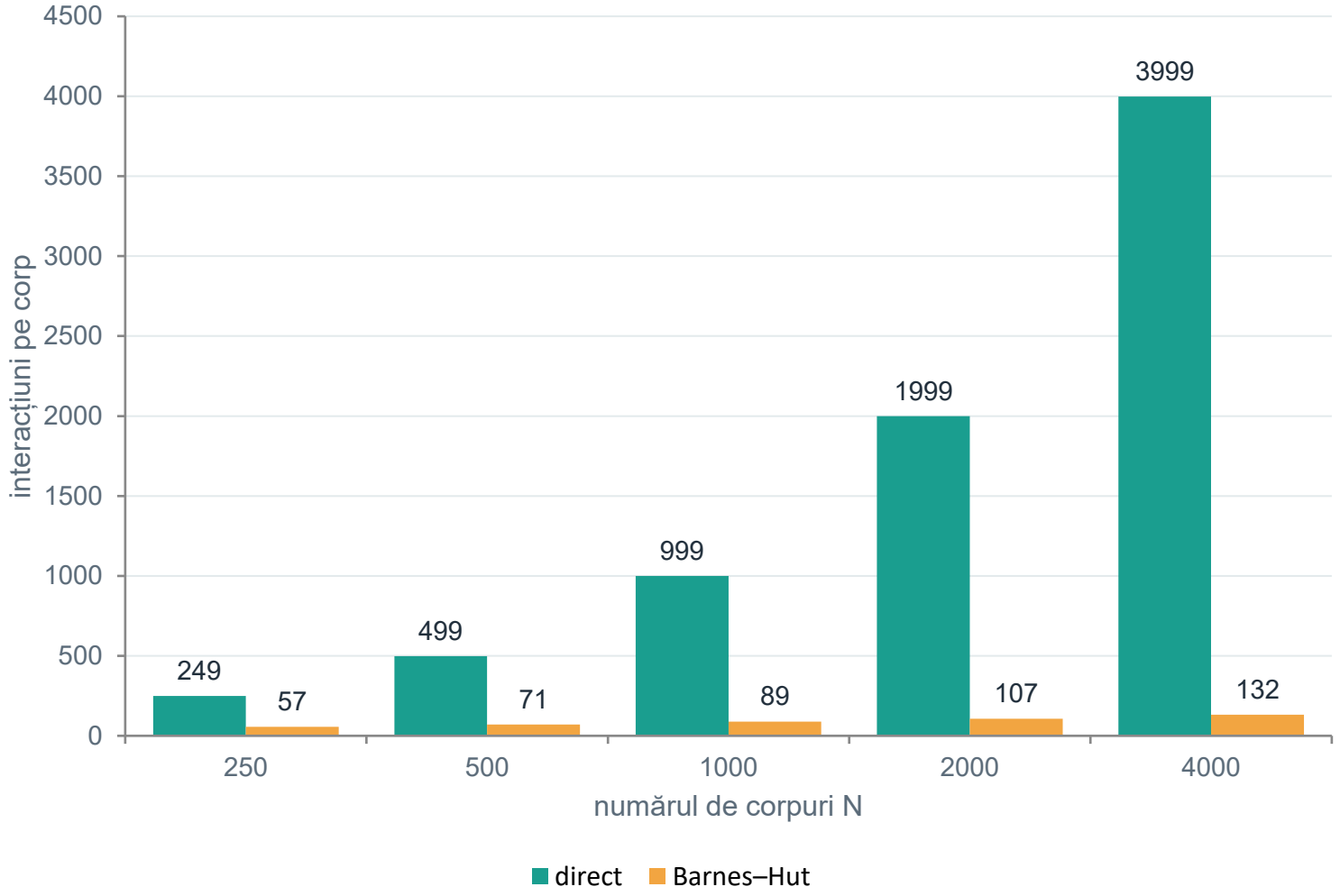
Arborele quadtree pentru două „galaxii” de 150 de stele



celulele se subdivid până la cel mult un corp; zonele dense au celule mici

- Inserarea unui corp: coborâm în cadranul potrivit (NV, NE, SV, SE) și subdivizăm când e ocupat
- Adâncimea arborelui ≈ $\log_4 N$ pentru distribuții uniforme
- Construcție: O(N log N); se refac la fiecare pas
- Același arbore servește și la detecția coliziunilor în jocuri

Barnes–Hut (θ = 0,5): interacțiuni calculate pentru fiecare corp



- N = 4 000: **131,5** interacțiuni în loc de 3 999 (~30× mai puține)
- Dublăm N: interacțiunile pe corp cresc cu o **constantă**, nu se dublează (log N)
- Eroarea forței (mediana): 0,6–1,2%
- Pentru 10^{10} corpuri câștigul ajunge la ~10^{7} ori

Complexitatea calculului forțelor: rezumat

Metoda	Cost pe pas	Când se folosește	Exemple
Toate perechile	$O(N^2)$	N mic ($\leq \sim 10^4$) sau verificare	cod de referință
Rază de tăiere + grilă	O(N)	forțe cu rază scurtă	Lennard-Jones, DEM, SPH
Barnes–Hut (arbore)	O(N log N)	gravitație, N mare	galaxii, cosmologie
Multipoli rapizi (FMM)	O(N)	gravitație / Coulomb, precizie mare	Greengard–Rokhlin, 1987
Particle-mesh (PME)	O(N log N)	electrostatică periodică	GROMACS, prin FFT

- Algoritmul bun câștigă **ordine de mărime**; hardware-ul mai rapid câștigă doar factori constanți
- FMM a fost inclus printre cei 10 algoritmi ai secolului XX (Computing in Science & Engineering, 2000)

Simulări de particule în practică

1 Cosmologie

Simulările Millennium (2005, 10^{10} particule) și urmașele lor: cum s-au format galaxiile din fluctuații mici.

2 Proteine

Anton (D. E. Shaw): hardware dedicat dinamicii moleculare, simulări de milisecunde ale pliării proteinelor.

3 Fluide SPH

Valuri, inundații, efecte vizuale: fluidul ca nor de particule, cu vecini găsiți prin grilă.

4 Jocuri

Faza „broad phase”: grile sau arbori elimină perechile îndepărtate; „narrow phase” calculează contactele.

5 Inginerie

DEM pentru silozuri, mori, transportoare: milioane de particule cu forțe de contact.

6 Biologie

Celule ca particule care se resping și se lipesc: creșterea țesuturilor și a tumorilor.

Verificare

? În Barnes–Hut **mărim** parametrul θ de la 0,5 la 1,0. Ce se întâmplă?

A simularea devine exactă

B mai multe celule sunt tratate ca un singur corp: mai rapid, mai puțin precis

C mai puține celule sunt aproximate: mai lent, mai precis

D costul devine $O(N^2)$

Performanță

De la cod la supercalculator

1. Bucle Python, NumPy, C++: aceeași sumă
2. Memorie partajată și distribuită
3. Descompunerea domeniului, legea lui Amdahl
4. GPU și supercalculatoare exascale

Suma a 10^6 numere: cinci implementări (Cernazean L2)

Implementarea	Noi (ms)	Raport față de NumPy	Cernazean (ms)
Python, citire liniară	33,6	124×	59,4
Python, citire aleatoare	184	681×	270
Python, funcția sum()	3,61	13×	6,54
NumPy, a.sum()	0,27	1×	0,634
C++, buclă liniară	–	–	0,478

- Același algoritm, diferențe de **sute de ori**, doar din implementare și din ordinea accesului la memorie
- Cernazean: scris naiv, C++ e de ~100 de ori mai rapid decât Python; cu NumPy, performanța devine **comparabilă**

De ce NumPy este rapid și de ce contează ordinea în memorie

- Bucla rulează în **cod compilat** (C), nu în interpretorul Python: fără verificări de tip la fiecare element
- Datele sunt **contigue** în memorie: procesorul le aduce în cache în blocuri de 64 de octeți
- Instrucțiuni **SIMD**: un singur ciclu procesează 4–16 numere deodată
- Accesul aleator ratează cache-ul: fiecare citire poate costa ~100 ns din memoria principală, față de ~1 ns din cache
- Regula practică: operații pe vectori întregi, acces în ordine, cât mai puține bucle Python

M

Același principiu, peste tot

Game of Life (~557×), LBM, perechile de particule: viteza vine din a lucra pe blocuri mari de date așezate continuu.

Memorie partajată și memorie distribuită

Memorie partajată (OpenMP, fire de execuție)

- Mai multe nuclee ale **aceluiași** calculator
- Toate văd aceeași memorie
- Simplu: paralelizăm bucla (de ex. #pragma omp parallel for)
- Limita: zeci – sute de nuclee
- Atenție la **condiții de cursă** când scriu în același loc

vs

Memorie distribuită (MPI)

- **Multe** calculatoare legate prin rețea
- Fiecare are memoria lui
- Datele se trimit explicit prin **mesaje**
- Scalează la milioane de nuclee
- Costul: comunicarea – trebuie redusă la minimum

Supercalculatoarele combină ambele: MPI între noduri, OpenMP sau GPU în interiorul fiecărui nod. În Python: multiprocessing, Numba, Dask, mpi4py.

Descompunerea domeniului: comunicare vs calcul

Bloc pe procesor	Celule de calculat	Celule de margine (halo)	Raport
10 × 10	100	40	40,0%
100 × 100	10 000	400	4,0%
1 000 × 1 000	1 000 000	4 000	0,4%

- Grila se împarte în **blocuri**, câte unul pe procesor; la fiecare pas, blocurile vecine schimbă doar **marginile**
- Calculul crește cu aria (n^2), comunicarea cu perimetrul (4n): blocurile mari sunt eficiente
- Palabos (Cernazean L2): abordare „multi-block” – eficientă dacă comunicarea $\ll$ calculul și încărcarea e **echilibrată**

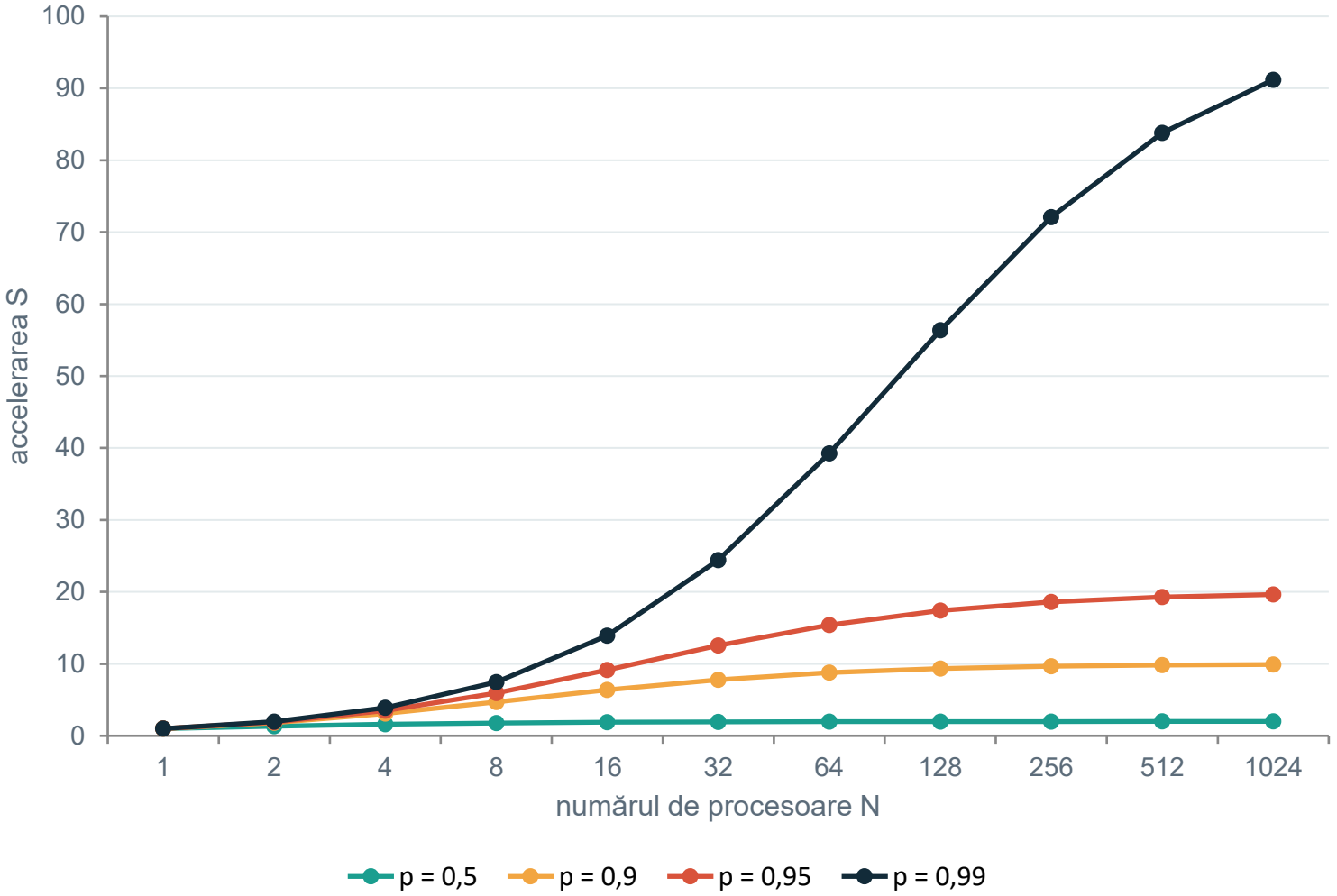
Legea lui Amdahl (1967)

$$S(N) = 1 / ((1 - p) + p / N)$$

p	fracțiunea din timpul de execuție care se poate paraleliza
N	numărul de procesoare
limita	pentru N foarte mare, S tinde la **1 / (1 – p)**: partea secvențială decide plafonul

Exemplu: p = 0,9 și N = 16 dau S = 1/(0,1 + 0,05625) ≈ 6,4; cu oricâte procesoare, S < 10. Legea lui Gustafson (1988) e mai optimistă: dacă mărim problema odată cu N, partea paralelă crește și ea.

Accelerarea după legea lui Amdahl



- p = 0,5: plafon **2**
- p = 0,9: 9,91 la 1 024 de procesoare (plafon 10)
- p = 0,99: 91,18 la 1 024 (plafon 100)
- Fiecare procent secvențial costă enorm la scară mare
- Axa X este logaritmică (dublări)

Ce se poate paraleliza și ce nu

Paralelizabil

- Celulele unui automat celular la același pas
- Forțele pe particule diferite
- Replicările Monte Carlo (S2, S6)
- Punctele unei diagrame (fiecare ρ, fiecare p)
- „Jenant de paralel”: zero comunicare

Greu sau imposibil de paralelizat

- Fibonacci: $F(i) = F(i - 1) + F(i - 2)$
- Pașii de timp: t + 1 cere pasul t
- Lista de evenimente din DES (S5)
- Actualizarea asincronă a unui CA
- Sumele globale: cer sincronizare

Paralelizăm în **spațiu** (celule, particule) și între **replicări** – rareori în timp. Simularea discretă pe evenimente paralelă este un domeniu de cercetare în sine (protocoale conservative și optimiste, Time Warp).

GPU: mii de nuclee simple pentru calcule regulate

- Cernazean (L2): un procesor modern ~2–3 TFLOPS; un GPU de consum ~**80 TFLOPS** (precizie simplă)
- GPU-ul rulează același program pe mii de fire: ideal pentru celule, particule, matrice
- Ce merge excelent: CA, LBM, perechi de particule, rețele neuronale; ce merge prost: ramificații multe, acces aleator, recursivitate
- Din Python: **CuPy** (NumPy pe GPU), **JAX**, **Numba** (kerneluri CUDA), PyTorch
- Costul ascuns: copierea datelor între CPU și GPU – datele trebuie să **rămână** pe GPU

P **Lista de verificare a performanței**

1. Măsoară (profiler). 2. Algoritmul. 3. Vectorizare. 4. Cod compilat (Numba). 5. Paralelism. 6. GPU. – în această ordine.

Supercalculatoare: lista TOP500 din iunie 2026

2,198 EFlop/s

LineShine (Shenzhen, China): primul loc, doar cu procesoare CPU

5

sisteme **exascale** (peste 10^{18} operații pe secundă): LineShine, El Capitan, Frontier, Aurora, JUPITER

1,0 EFlop/s

JUPITER (Jülich, Germania): primul sistem exascale al Europei (noiembrie 2025)

- El Capitan 1,809, Frontier 1,353, Aurora 1,012 EFlop/s (SUA); performanța cumulată a listei: 18,73 EFlop/s
- Un laptop: ~1 TFLOPS – un supercalculator exascale este de ~1 milion de ori mai rapid; cursul Cernazean (2024) menționa „~1 EFlop/s” – depășit între timp

Verificare

? Un program are **90%** din timp paralelizabil. Care este accelerarea **maximă** posibilă, cu oricâte procesoare?

A de 9 ori

B de 10 ori

C de 90 de ori

D nelimitată

Rezumatul săptămânii

- ✓ Automate celulare: reguli **locale**, comportament **global**; 256 de reguli elementare, Game of Life, emergență și ireductibilitate
- ✓ Modele CA ale realității: trafic **Nagel–Schreckenberg**, percolare (pc ≈ 0,593), difuzie, **Lattice Boltzmann**
- ✓ Particule: Newton + potențiale; **velocity Verlet** – ieftină, simplectică, eroare de energie mărginită
- ✓ Costul forțelor: $O(N^2)$; rază de tăiere + grilă O(N); **Barnes–Hut** O(N log N)
- ✓ Performanță: NumPy ≈ C++, descompunerea domeniului, **Amdahl** (plafon 1/(1 − p)), GPU, exascale

Săptămâna viitoare

Modelarea bazată pe agenți și sisteme complexe

- Agenți: percepție, decizie, acțiune, interacțiune
- Modele clasice: Boids, segregarea Schelling, furnicile
- Protocolul ODD și agenții generativi cu LLM

Celulele de azi aveau reguli fixe și nu se mișcau. Ce se întâmplă când fiecare individ percepe, decide și se deplasează – și cum ajunge segregat un oraș în care nimeni nu vrea segregare?

Bibliografie pentru săptămâna 8

- Prof. Cernazean – Modelling and Simulation, Lecture 2 (HPC), Lecture 4 (cellular automata), Lecture 5 (lattice Boltzmann), Lecture 6 (particles) (materiale de curs)
- B. Chopard, M. Droz – Cellular Automata Modeling of Physical Systems, Cambridge University Press, 1998
- S. Wolfram – A New Kind of Science, Wolfram Media, 2002
- K. Nagel, M. Schreckenberg – A cellular automaton model for freeway traffic, J. Phys. I France 2, 1992
- T. Krüger et al. – The Lattice Boltzmann Method: Principles and Practice, Springer, 2017
- M. P. Allen, D. J. Tildesley – Computer Simulation of Liquids, ed. 2, Oxford University Press, 2017
- J. Barnes, P. Hut – A hierarchical O(N log N) force-calculation algorithm, Nature 324, 1986
- TOP500 – lista din iunie 2026, top500.org
- Prezentare realizata cu ajutorul Claude Fable 5.1 și Claude Opus 5.5